TÀI LIỆU ĐẶC TẢ NGHIỆP VỤ VÀ YÊU CẦU KỸ THUẬT
HỆ THỐNG ĐẶT PHÒNG KHÁCH SẠN / CĂN HỘ

Phạm vi: nghiệp vụ đặt phòng, tồn kho, thanh toán qua Fake Bank, quản trị, bảo mật và vận hành

Tài liệu đặc tả độc lập dùng làm cơ sở thống nhất yêu cầu, triển khai, kiểm thử và phối hợp phát triển.

1. Tổng quan và phạm vi

Hệ thống cho phép khách tìm kiếm, xem giá theo từng đêm, giữ phòng, thanh toán qua Fake Bank, hủy và theo dõi thời gian lưu trú; chủ cơ sở lưu trú quản lý phòng, lịch mở bán và xử lý đặt phòng; bộ phận hỗ trợ và quản trị quản lý theo phạm vi quyền. PostgreSQL là nguồn dữ liệu nghiệp vụ có tính quyết định; Redis tăng tốc truy vấn và hỗ trợ giới hạn tần suất; MinIO lưu tệp ảnh;

Prometheus/Grafana phục vụ theo dõi hệ thống.

Các yêu cầu dưới đây mô tả hành vi bắt buộc theo góc nhìn nghiệp vụ, các trường hợp ngoại lệ, dữ liệu liên quan, điểm đồng bộ và hệ quả kỹ thuật. Giá, hủy và tồn kho phải dựa trên snapshot dữ liệu tại thời điểm đặt, không diễn giải lại lịch sử theo cấu hình mới.

Trong phạm vi: Tài khoản và phân quyền; cơ sở/phòng/giá/tiện ích/ảnh; tìm kiếm; đặt và giữ chỗ; payment/refund mô phỏng; hủy; check-in/out; đánh giá; thông báo nội bộ; báo cáo; audit; cache, giám sát.

1.1. Vai trò và ranh giới truy cập

Vai trò	Nghiệp vụ được phép	Giới hạn bắt buộc
GUEST	Tìm kiếm, xem chi tiết/các chính sách công khai	Không tạo booking; không xem thông tin cá nhân
CUSTOMER	Quản lý hồ sơ, đặt/hủy phòng, xem lịch sử, đánh giá	Chỉ dữ liệu thuộc chính mình
HOST	Quản lý property, loại phòng, phòng, giá, ảnh; check-in/out	Chỉ property có owner_id hợp lệ
SUPPORT	Tra cứu xử lý yêu cầu hỗ trợ theo quyền được cấp	Không tự thay giá/thanh toán; hành động có audit
ADMIN	Quản trị tài khoản, role, cấu hình, tra cứu audit	Tác vụ nhạy cảm có xác thực, lý do và lịch sử

1.2. Thuật ngữ, chuẩn tính ngày và nguyên tắc bất biến

- Property = cơ sở lưu trú; RoomType = loại phòng; RoomUnit/Room = phòng vật lý; Booking = đơn đặt phòng; Hold = giữ chỗ có thời hạn; Payment/Refund = thanh toán/hoàn tiền mô phỏng.
- Khoảng lưu trú là [checkIn, checkOut): tính đêm nhận phòng nhưng không tính đêm trả phòng; checkOut phải sau checkIn; múi giờ nghiệp vụ lấy theo property.
- Một phòng vật lý không được có hai booking đang chiếm dụng có khoảng ngày giao nhau; mọi nhánh cập nhật booking phải giữ invariant này kể cả khi chạy đồng thời.

- Giá từng đêm và chính sách hủy được lưu thành snapshot trong booking; mọi khoản tiền dùng NUMERIC/BigDecimal và currency thống nhất (VND).
- Trạng thái thanh toán và trạng thái booking là hai vòng đời riêng; callback đến trễ không được tự động biến booking đã hết hạn thành CONFIRMED.
- Mọi thao tác đọc/ghi kiểm tra RBAC + phạm vi dữ liệu; không tin owner_id, role, số tiền, payment_status hay trạng thái gửi từ trình duyệt.
- Không xóa cứng đối tượng đã có lịch sử đặt phòng. Mọi chuyển trạng thái có nguồn gốc, thời gian, actor và lý do nếu cần.

2. Danh mục use case và quan hệ phụ thuộc

Mã	Nghiệp vụ	Tác nhân	Phân hệ
UC-01	Đăng ký và xác minh tài khoản	GUEST	Identity
UC-02	Đăng nhập, làm mới và đăng xuất	CUSTOMER/HOST/ADMIN	Identity
UC-03	Quản lý hồ sơ và phân quyền	CUSTOMER/ADMIN	Identity
UC-04	Tạo và xuất bản cơ sở lưu trú	HOST	Property
UC-05	Quản lý loại phòng, phòng, giá và lịch bán	HOST	Catalog
UC-06	Upload và quản lý ảnh	HOST	Media
UC-07	Tìm kiếm và lọc phòng	GUEST/CUSTOMER	Search
UC-08	Xem chi tiết và báo giá	GUEST/CUSTOMER	Pricing
UC-09	Tạo booking và giữ chỗ	CUSTOMER	Booking/Inventory
UC-10	Thanh toán mô phỏng và callback	CUSTOMER/Simulator	Payment
UC-11	Job hết hạn giữ chỗ	SCHEDULER	Booking/Inventory
UC-12	Hủy booking và tính phí	CUSTOMER/SUPPORT	Booking
UC-13	Hoàn tiền mô phỏng và đối soát	SYSTEM/SUPPORT	Refund
UC-14	Chủ cơ sở xử lý đặt phòng	HOST	Booking
UC-15	Nhận phòng, trả phòng, hoàn tất	HOST	Stay
UC-16	Đánh giá và kiểm duyệt phản hồi	CUSTOMER/SUPPORT	Review
UC-17	Thông báo trong hệ thống	SYSTEM	Notification

UC-18	Báo cáo doanh thu và tình trạng phòng	HOST/ADMIN	Reporting
UC-19	Audit và hỗ trợ tra cứu sự cố	SUPPORT/ADMIN	Audit
UC-20	Quản trị danh mục và khóa tài khoản	ADMIN	Administration
UC-21	Thống kê dashboard tổng quan theo phạm vi dữ liệu	HOST/ADMIN	Analytics
UC-22	Thống kê đặt phòng đa chiều và so sánh kỳ	HOST/ADMIN	Analytics
UC-23	Thống kê doanh thu, dòng tiền, hoàn tiền	HOST/ADMIN	Analytics
UC-24	Thống kê công suất phòng, tồn kho, giá theo ngày	HOST/ADMIN	Analytics
UC-25	Thống kê hủy phòng, lỗi thanh toán và phễu đặt phòng	HOST/ADMIN	Analytics
UC-26	Truy vấn thống kê động, export và quản lý cache Redis	HOST/ADMIN	Analytics
UC-27	Mở hội thoại khách hàng – chủ cơ sở	CUSTOMER/HOST	Messaging
UC-28	Nhắn tin, phản hồi và phản ánh sự cố lưu trú	CUSTOMER/HOST/SUPPORT	Messaging
UC-29	Hẹn check-in, xác nhận và xử lý vắng mặt qua hội thoại	CUSTOMER/HOST	Messaging/Stay
UC-30	Cấu hình chính sách hệ thống và độ dài mật khẩu	ADMIN	Configuration
UC-31	Quản trị mẫu email và thay thế biến nội dung	ADMIN	Email
UC-32	Email giao dịch: tạo tài khoản, booking, thanh toán, hoàn tiền	SYSTEM	Email

UC-33	Nhắc lịch check-in bằng email theo cấu hình	SYSTEM	Email/Scheduler
UC-34	Theo dõi email lỗi, retry, chống gửi trùng	SYSTEM/SUPPORT	Email Operations
UC-35	Fake Bank: số dư giả lập, debit, webhook, đối soát	SYSTEM/ADMIN	Fake Bank
UC-36	Bù trừ giao dịch, hủy thanh toán, hoàn tiền liên hệ thống	SYSTEM/SUPPORT	Payment/Refund

Quy trình xuyên suốt: UC-01/02 → UC-07/08 → UC-09 → UC-10 → UC-14/15 → UC-16. Ngoại lệ: UC-09 → UC-11 khi giữ chỗ hết hạn, hoặc UC-10 → UC-12/13 khi hủy và hoàn tiền. UC-17/18/19 theo dõi các thay đổi phát sinh trong nhiều use case.

3. Đặc tả chi tiết từng use case

Mỗi use case được trình bày theo một cấu trúc thống nhất: tác nhân, tiền điều kiện, sự kiện kích hoạt, luồng chuẩn theo thứ tự thực hiện, ngoại lệ, quy tắc bất biến, kết quả cuối cùng và kinh nghiệm kỹ thuật có thể đúc kết từ việc triển khai. Các bước là quy trình nghiệp vụ, không quy định bắt buộc một tên class cụ thể.

UC-01. Đăng ký và xác minh tài khoản

Tác nhân: GUEST

Tiền điều kiện: Email chưa tồn tại; hệ thống cho phép đăng ký công khai.

Kích hoạt: Khách gửi form đăng ký.

Luồng nghiệp vụ chính

Bước 01. Khách nhập email, mật khẩu, họ tên, số điện thoại (nếu bắt buộc) và xác nhận điều khoản.

Bước 02. API chuẩn hóa email (trim/lowercase), kiểm tra độ dài, định dạng, mật khẩu và dữ liệu bắt buộc.

Bước 03. Hệ thống tìm email đã có; đồng thời dựa vào unique constraint tại PostgreSQL để chặn hai request đăng ký cùng email.

Bước 04. Mật khẩu được hash bằng PasswordEncoder phù hợp; không ghi password hoặc token vào log.

Bước 05. Tạo User trạng thái PENDING_VERIFICATION với role CUSTOMER mặc định; không chấp nhận role tùy ý từ request.

Bước 06. Sinh mã xác minh ngẫu nhiên, chỉ lưu bản băm/giá trị an toàn với expires_at, hạn chế số lần dùng.

Bước 07. Gửi thông báo xác minh qua bộ giả lập thông báo; không làm thất bại giao dịch user chỉ do kênh gửi tạm lỗi (có trạng thái gửi lại).

Bước 08. Khách gửi mã xác minh; server kiểm tra user, mã, hạn dùng và số lần thử.

Bước 09. Cập nhật tài khoản ACTIVE, thu hồi mã đã dùng và ghi dấu thời điểm xác minh.

Luồng thay thế, ngoại lệ và cách xử lý

- Email trùng → EMAIL_ALREADY_EXISTS; xung đột unique được ánh xạ thành lỗi nghiệp vụ.
- Mã sai/hết hạn → VERIFICATION_INVALID/EXPIRED; giới hạn resend và số lần thử.
- Lỗi gửi thông báo → lưu tác vụ chờ gửi lại, không tạo tài khoản thứ hai.

Quy tắc nghiệp vụ / ràng buộc

- Email unique không phân biệt chữ hoa/chữ thường; không tiết lộ tài khoản tại endpoint nhạy cảm khi cần chống enumeration.
- Không cho gửi trạng thái ACTIVE, userId, role từ bên ngoài.

Kết quả sau cùng: User hợp lệ; trạng thái ACTIVE; có lịch sử xác minh.

Kinh nghiệm kỹ thuật đúc kết: Unique constraint vẫn cần dù đã kiểm tra exists; xác minh phải chống dùng lại token và xử lý retry thông báo.

UC-02. Đăng nhập, làm mới token và đăng xuất

Tác nhân: CUSTOMER/HOST/SUPPORT/ADMIN

Tiền điều kiện: Tài khoản ACTIVE; thông tin đăng nhập hợp lệ.

Kích hoạt: Người dùng thực hiện xác thực.

Luồng nghiệp vụ chính

Bước 01. Client gửi email/mật khẩu qua HTTPS; backend chuẩn hóa email và áp dụng rate-limit theo user/IP.

Bước 02. AuthenticationManager/Provider tra cứu user, kiểm tra hash mật khẩu và trạng thái tài khoản.

Bước 03. Đọc role/permission hiệu lực; sinh access JWT thời hạn ngắn với subject, quyền và mốc cấp token.

Bước 04. Nếu dùng refresh token: sinh chuỗi ngẫu nhiên, lưu hash, hạn dùng, thiết bị/phạm vi, trả token cho client bằng cơ chế an toàn.

Bước 05. Client gửi access token trong mỗi request; filter xác minh chữ ký, hạn dùng, issuer/audience (nếu cấu hình), trạng thái thu hồi theo chính sách.

Bước 06. Khi access token hết hạn, client gửi refresh token; server kiểm tra hash, hạn và trạng thái, sau đó rotate refresh token.

Bước 07. Đăng xuất: vô hiệu refresh token hiện tại; khi cần thu hồi access tức thời, áp dụng token version/denylist ngắn hạn và kiểm tra khi request.

Bước 08. Theo dõi login success/failure không kèm password/token và giữ correlation ID.

Luồng thay thế, ngoại lệ và cách xử lý

- Sai thông tin/khóa tài khoản → 401 theo chính sách; không tiết lộ mật khẩu đúng hay email tồn tại.
- Refresh token đã dùng lại sau rotation → từ chối và có thể thu hồi phiên liên quan.
- JWT hợp lệ nhưng role bị thu hồi → áp dụng chính sách làm mới authorization/token version.

Quy tắc nghiệp vụ / ràng buộc

- 401 = chưa xác thực; 403 = đã xác thực nhưng thiếu quyền.
- Không đưa thông tin nhạy cảm vào JWT; ký và quản lý secret/keys ngoài source code.

Kết quả sau cùng: Phiên đăng nhập hợp lệ hoặc bị vô hiệu theo yêu cầu.

Kinh nghiệm kỹ thuật đúc kết: Phân biệt authentication và authorization; JWT stateless không có nghĩa tài khoản khóa sẽ bị chặn tức thì nếu không có chính sách thu hồi.

UC-03. Quản lý hồ sơ người dùng và phạm vi quyền

Tác nhân: CUSTOMER/ADMIN

Tiền điều kiện: Đã đăng nhập; admin phải có quyền USER_MANAGE khi can thiệp người khác.

Kích hoạt: Yêu cầu xem/sửa hồ sơ hoặc cấp quyền.

Luồng nghiệp vụ chính

Bước 01. Đọc userId từ principal đã xác thực, không lấy từ body tùy ý cho endpoint /me.

Bước 02. Tải hồ sơ bằng projection/DTO, loại bỏ passwordHash, refresh token và trường quản trị.

Bước 03. Người dùng sửa họ tên, số điện thoại, ảnh đại diện và các trường được phép.

Bước 04. Validate rồi cập nhật các cột cho phép; lưu updated_at và version để phát hiện ghi đè.

Bước 05. Đổi mật khẩu yêu cầu xác minh mật khẩu cũ hoặc luồng reset tin cậy; vô hiệu phiên theo chính sách.

Bước 06. Admin cấp/thu role theo API riêng: kiểm tra quyền, không cho cấp quyền vượt chính sách được ủy quyền.

Bước 07. Ghi audit trước/sau bằng danh sách trường thay đổi đã che dữ liệu nhạy cảm.

Bước 08. Request tiếp theo phản ánh phạm vi quyền mới theo token/session strategy.

Luồng thay thế, ngoại lệ và cách xử lý

- Chỉnh sửa hồ sơ người khác → 403/404 theo chính sách ẩn tài nguyên.
- Concurrent update → VERSION_CONFLICT và yêu cầu tải lại dữ liệu.
- Body cố gắng sửa role/owner/status → bỏ qua không đủ; DTO không cho nhận và kiểm thử mass assignment.

Quy tắc nghiệp vụ / ràng buộc

- RBAC phải kết hợp data scope; một quyền UPDATE_USER không mặc nhiên được sửa bất kỳ tài khoản nào.

Kết quả sau cùng: Hồ sơ được cập nhật hoặc từ chối rõ ràng; lịch sử quyền có thể truy vết.

Kinh nghiệm kỹ thuật đúc kết: DTO riêng cho từng use case chống mass assignment; security không nên chỉ nằm ở HTTP/controller.

UC-04. Chủ cơ sở tạo và xuất bản property

Tác nhân: HOST

Tiền điều kiện: HOST ACTIVE, có quyền PROPERTY_CREATE; danh mục địa điểm hợp lệ.

Kích hoạt: HOST tạo thông tin cơ sở.

Luồng nghiệp vụ chính

Bước 01. HOST khởi tạo property ở DRAFT, hệ thống gán ownerId từ JWT và timezone theo địa điểm.

Bước 02. HOST nhập tên, địa chỉ, mô tả, vị trí, giờ check-in/out, chính sách hủy và thông tin liên hệ công khai.

Bước 03. Server kiểm tra trường bắt buộc, giá trị tọa độ, timezone và chính sách hợp lệ.

Bước 04. HOST tạo ít nhất một room type, room vật lý và bảng giá hợp lệ theo UC-05.

Bước 05. HOST tải ảnh theo UC-06 và chọn ảnh đại diện hợp lệ.

Bước 06. HOST yêu cầu xuất bản; service tải toàn bộ dữ liệu liên quan và đánh giá điều kiện publish.

Bước 07. Nếu đầy đủ, cập nhật DRAFT → PUBLISHED cùng phiên bản; ghi audit và phát hành tác vụ xóa cache chi tiết/tìm kiếm.

Bước 08. API trả trạng thái cuối cùng; property chỉ xuất hiện trong kết quả tìm kiếm sau khi đã PUBLISHED.

Luồng thay thế, ngoại lệ và cách xử lý

- Thiếu phòng, giá, chính sách, ảnh → PROPERTY_NOT_READY kèm danh sách thiếu.
- HOST sửa property của người khác → FORBIDDEN_SCOPE.
- Đang có booking tương lai, yêu cầu ARCHIVED → không xóa dữ liệu lịch sử; áp dụng quy trình dừng bán.

Quy tắc nghiệp vụ / ràng buộc

- PUBLISHED → PAUSED chặn bán mới, không tự hủy booking cũ.
- Không xóa cứng property đã phát sinh giao dịch.

Kết quả sau cùng: Property DRAFT/PUBLISHED; phiên bản thay đổi rõ ràng.

Kinh nghiệm kỹ thuật đúc kết: Publish là một use case kiểm tra tính nhất quán xuyên nhiều bảng, không đơn thuần cập nhật cờ isPublished.

UC-05. Quản lý loại phòng, phòng vật lý, giá và lịch mở bán

Tác nhân: HOST

Tiền điều kiện: Property thuộc HOST; property chưa ARCHIVED.

Kích hoạt: HOST tạo/sửa tồn kho thương mại.

Luồng nghiệp vụ chính

Bước 01. HOST tạo loại phòng với sức chứa, tiện nghi, loại giường, mô tả và quy định số khách.

Bước 02. Hệ thống validate sức chứa dương và giá trị danh mục; lưu RoomType.

Bước 03. HOST thêm các RoomUnit với mã phòng unique trong property; chọn ACTIVE/MAINTENANCE theo tình trạng.

Bước 04. HOST thiết lập room_rate theo ngày/khoảng ngày và currency; server xử lý ngày biên [from,to).

Bước 05. Kiểm tra giá không âm, không tạo nhiều giá hiệu lực xung đột cho cùng đối tượng/ngày.

Bước 06. Khi HOST sửa giá, server cập nhật bảng giá bán tương lai và tăng cache version/invalidate.

Bước 07. Khi HOST đóng bán hoặc giảm sức chứa, hệ thống kiểm tra booking CONFIRMED/CHECKED_IN và hold còn hiệu lực trong tương lai.

Bước 08. Nếu có xung đột, từ chối thao tác hoặc yêu cầu xử lý booking trước; ghi audit giá trước/sau.

Bước 09. Tìm kiếm mới dùng giá mới; booking đã tạo giữ nguyên booking_nights snapshot.

Luồng thay thế, ngoại lệ và cách xử lý

- Mã phòng trùng → ROOM_CODE_DUPLICATE.
- Ngày thiếu giá → không cho chốt booking ở giai đoạn báo giá; có thể hiển thị “chưa có giá”.
- Đóng phòng đang có booking → ROOM_HAS_FUTURE_BOOKINGS; không xóa quan hệ lịch sử.

Quy tắc nghiệp vụ / ràng buộc

- Quy tắc bán dựa vào room physical/inventory chính, không dựa vào số phòng hiển thị ở client.

Kết quả sau cùng: Danh mục phòng/giá hợp lệ và có lịch sử cấu hình.

Kinh nghiệm kỹ thuật đúc kết: Phân biệt catalog với inventory đang chiếm; sửa giá không được làm biến đổi số tiền của đơn cũ.

UC-06. Upload, duyệt và xóa ảnh MinIO

Tác nhân: HOST

Tiền điều kiện: Tài nguyên property/room thuộc HOST; bucket được backend quản trị.

Kích hoạt: HOST thêm/xóa ảnh cơ sở hoặc phòng.

Luồng nghiệp vụ chính

Bước 01. HOST chọn room/property và gửi yêu cầu tạo lượt upload với filename, size, content-type dự kiến.

Bước 02. Backend xác thực quyền sở hữu; giới hạn dung lượng, phần mở rộng và số ảnh trên tài nguyên.

Bước 03. Tạo media_asset trạng thái PENDING, sinh object key từ UUID và tiền tố nội bộ; không tin đường dẫn client.

Bước 04. Backend cấp presigned upload URL ngắn hạn hoặc nhận stream trực tiếp theo cấu hình.

Bước 05. Client upload; backend thực hiện bước complete/finalize, HEAD object và kiểm tra thực tế size/MIME/checksum theo chính sách.

Bước 06. Đổi media_asset thành READY, lưu metadata, liên kết ảnh với room/property và thứ tự hiển thị.

Bước 07. Tạo URL xem có hạn cho bucket private; ảnh công khai chỉ dùng bucket/policy được kiểm soát.

Bước 08. Khi xóa ảnh, xóa liên kết/đánh dấu DELETING, thực hiện xóa object bất đồng bộ; dọn object mồ côi theo job.

Bước 09. Invalidate cache property/room để ảnh mới xuất hiện nhất quán.

Luồng thay thế, ngoại lệ và cách xử lý

- File giả mạo MIME/size vượt giới hạn → MEDIA_INVALID.
- Upload MinIO thành công nhưng DB lỗi → orphan cleanup dựa trên PENDING TTL/danh sách object.
- Upload fail/chưa finalize → media PENDING_EXPIRED và dọn metadata theo chính sách.

Quy tắc nghiệp vụ / ràng buộc

- Binary ở MinIO, metadata ở PostgreSQL; không giữ transaction DB khi truyền tệp dung lượng lớn.

Kết quả sau cùng: Ảnh READY hiển thị theo đúng quyền; object lỗi được dọn an toàn.

Kinh nghiệm kỹ thuật đúc kết: Phân biệt transaction DB với side-effect ở object storage; presigned URL không phải token quyền vĩnh viễn.

UC-07. Tìm kiếm phòng theo tiêu chí và thời gian

Tác nhân: GUEST/CUSTOMER

Tiền điều kiện: Property PUBLISHED; dữ liệu địa điểm/loại phòng sẵn sàng.

Kích hoạt: Người dùng tìm phòng trống.

Luồng nghiệp vụ chính

Bước 01. Nhận địa điểm, ngày nhận/trả, số khách, tiện ích, mức giá, loại hình lưu trú, sort, page/size.

Bước 02. Validate checkIn trước checkOut, ngày không thuộc quá khứ theo múi giờ cơ sở, số khách > 0, khoảng giá đúng và size giới hạn.

Bước 03. Chuẩn hóa bộ lọc; tạo cache key có ngày, khách, giá, tiện ích, sort, page, version chính sách.

Bước 04. Tra Redis nếu bật cache; cache hit trả kết quả sơ bộ còn trong TTL, không cam kết giữ phòng.

Bước 05. Nếu miss, query PostgreSQL chỉ tìm property PUBLISHED, room ACTIVE, sức chứa phù hợp và các điều kiện lọc.

Bước 06. Loại room có booking chiếm dụng giao [checkIn, checkOut); xét cả hold chưa hết hạn và booking CONFIRMED/CHECKED_IN.

Bước 07. Tính giá đại diện từ rates từng đêm; nếu không đủ giá cho các đêm cần có quy tắc loại bỏ.

Bước 08. Trả projection phân trang ổn định theo sort whitelist cộng khóa id; tránh N+1 khi nạp ảnh/tiện ích.

Bước 09. Cache kết quả TTL ngắn, trả danh sách, tổng phần tử và phân trang.

Luồng thay thế, ngoại lệ và cách xử lý

- Không có phòng → kết quả rỗng 200, không phải lỗi hệ thống.
- Cache stale sau sửa giá/tồn kho → khi tạo booking vẫn xác minh giá và availability ở DB.
- Query quá phức tạp/size quá lớn → VALIDATION_ERROR và giới hạn tài nguyên.

Quy tắc nghiệp vụ / ràng buộc

- Search chỉ là thông tin tham khảo; bước giữ phòng là nơi bắt buộc kiểm tra tồn kho lại.

Kết quả sau cùng: Danh sách phòng và giá hợp lệ theo thời điểm đọc.

Kinh nghiệm kỹ thuật đúc kết: Tối ưu truy vấn động bằng Criteria/Specification/EntityManager, EXPLAIN ANALYZE, composite index và query projection.

UC-08. Xem chi tiết phòng và nhận báo giá

Tác nhân: GUEST/CUSTOMER

Tiền điều kiện: Room thuộc property PUBLISHED, hiển thị công khai.

Kích hoạt: Người dùng xem phòng hoặc chọn ngày để báo giá.

Luồng nghiệp vụ chính

Bước 01. Client gửi propertyId/roomTypeId, tùy chọn checkIn/checkOut và guestCount.

Bước 02. Backend tải mô tả, địa điểm, tiện ích, sức chứa, ảnh READY, chính sách nhận/trả phòng và đánh giá hợp lệ.

Bước 03. Nếu chưa nhập ngày, hiển thị giá tham khảo và giải thích giá sẽ được xác định theo ngày.

Bước 04. Nếu có ngày, validate thời gian lưu trú và sức chứa theo room type.

Bước 05. Đọc room_rate cho từng stay_date trong [checkIn,checkOut); xác minh đủ dữ liệu giá.

Bước 06. Tính subtotal = tổng nightly_rate; phụ phí/thuế nếu có được hiển thị tách riêng theo cấu hình.

Bước 07. Tính tổng bằng BigDecimal, làm tròn theo currency; trả quote có thời hạn ngắn và priceVersion (nếu dùng).

Bước 08. Trả chính sách hủy có thời điểm hiệu lực, không tiết lộ dữ liệu liên lạc riêng tư.

Luồng thay thế, ngoại lệ và cách xử lý

- Thiếu giá của một đêm → RATE_NOT_CONFIGURED; không giả định base rate.
- Giá thay đổi giữa quote và booking → booking tính lại phía server, client được thông báo PRICE_CHANGED nếu khác.
- Phòng không còn khả dụng → hiển thị trạng thái tham khảo, không giữ chỗ.

Quy tắc nghiệp vụ / ràng buộc

- Quote không phải cam kết tồn kho; chỉ booking đã commit mới được giữ chỗ.

Kết quả sau cùng: Trang chi tiết và báo giá có thể giải thích theo từng đêm.

Kinh nghiệm kỹ thuật đúc kết: Thiết kế quote/booking giá bất biến; tránh lỗi số thực và timezone khi tính đêm.

UC-09. Tạo booking và giữ phòng có hạn

Tác nhân: CUSTOMER

Tiền điều kiện: Khách ACTIVE; phòng mở bán, còn tồn; bảng giá và chính sách đầy đủ.

Kích hoạt: Khách xác nhận đặt phòng.

Luồng nghiệp vụ chính

Bước 01. Client gửi roomId, checkIn, checkOut, guestCount, idempotencyKey; không tin totalAmount ở client.

Bước 02. Backend kiểm tra JWT, xác thực khách, validate ngày/sức chứa và điều kiện property PUBLISHED, room ACTIVE.

Bước 03. Kiểm tra idempotencyKey (scope theo user/endpoint) + requestHash; cùng key/payload trả kết quả cũ, cùng key/khác payload bị từ chối.

Bước 04. Bắt đầu transaction ngắn tại application service; khóa room hoặc dùng exclusion constraint/conditional inventory update theo mô hình thống nhất.

Bước 05. Đọc các booking/hold đang chiếm; kiểm tra không có khoảng ngày giao nhau dưới cơ chế khóa/constraint thực sự.

Bước 06. Đọc giá và chính sách hiện hành; tính tiền phía server theo từng đêm, phụ phí, thuế bằng BigDecimal.

Bước 07. Tạo booking PENDING_PAYMENT với expires_at; tạo booking_nights và snapshot chính sách hủy; gắn room allocation/hold.

Bước 08. Ghi idempotency record, booking_status_history và outbox notification trong cùng transaction.

Bước 09. Commit transaction rồi trả bookingId, totalAmount, expiresAt và hướng dẫn thanh toán.

Bước 10. Nếu client retry vì timeout, cùng idempotencyKey nhận booking cũ; không chiếm kho hai lần.

Luồng thay thế, ngoại lệ và cách xử lý

- Hai khách đặt cùng phòng/đêm → chỉ một transaction giữ chỗ thành công; bên còn lại ROOM_UNAVAILABLE.
- Lỗi chèn booking_nights → rollback booking + hold + idempotency record.
- Chờ lock quá lâu/deadlock → retry có giới hạn toàn use case nếu idempotent, không trả HTTP 500 mơ hồ.
- Thay đổi giá khi đang đặt → xử lý theo snapshot trong transaction, trả PRICE_CHANGED nếu lệch quote đã xác nhận.

Quy tắc nghiệp vụ / ràng buộc

- Không áp dụng SELECT rồi INSERT đơn lẻ như bảo đảm tồn kho.
- Redis lock không phải nguồn bảo đảm duy nhất; PostgreSQL có constraint/locking cuối cùng.

Kết quả sau cùng: Một booking PENDING_PAYMENT duy nhất giữ chỗ hợp lệ; lịch sử và tổng tiền nhất quán.

Kinh nghiệm kỹ thuật đúc kết: Ranh giới transaction, race condition, pessimistic/optimistic lock, idempotency và rollback toàn khối.

UC-10. Thanh toán qua Fake Bank, giao dịch lặp và xử lý lỗi liên hệ thống

Tác nhân: CUSTOMER, dịch vụ Booking và Fake Bank độc lập. Fake Bank có số dư tài khoản giả lập, giao dịch debit/credit, trạng thái giao dịch và webhook; tuyệt đối không sử dụng thẻ/ngân hàng thật. Hệ thống phải xử lý timeout, request lặp, webhook trùng, lỗi database, callback muộn và hoàn tiền bù trừ.

Tiền điều kiện: Booking PENDING_PAYMENT và có số tiền snapshot; Fake Bank bật ở môi trường cho phép.

Kích hoạt: Khách chọn thanh toán và hệ thống nhận webhook.

Luồng nghiệp vụ chính

Bước 01. Client yêu cầu khởi tạo thanh toán cho booking thuộc chính mình; service kiểm tra trạng thái và hạn giữ chỗ.

Bước 02. Tạo payment INITIATED với amount/currency đúng booking; dùng idempotencyKey riêng của payment.

Bước 03. Gửi yêu cầu cho Fake Bank ngoài transaction DB; Fake Bank trả providerReference và trạng thái xử lý.

Bước 04. Fake Bank phát callback success/failure/timeout theo các tình huống cấu hình.

Bước 05. Webhook kiểm tra chữ ký giả lập, timestamp/nonce và providerReference duy nhất; không tin trạng thái client tự báo.

Bước 06. Service bắt đầu transaction và khóa payment + booking theo một thứ tự cố định.

Bước 07. Nếu callback thành công và hold còn hợp lệ: payment → SUCCEEDED, booking → CONFIRMED, giữ allocation dưới dạng đã xác nhận.

Bước 08. Nếu thất bại: payment → FAILED, booking giữ PENDING_PAYMENT tới khi hết hạn hoặc chuyển trạng thái phù hợp theo chính sách.

Bước 09. Ghi lịch sử, outbox notification; commit và trả webhook thành công để hạn chế gửi lặp.

Bước 10. Chạy job đối soát để xử lý payment PROCESSING/TIMED_OUT hoặc callback bị mất.

Luồng thay thế, ngoại lệ và cách xử lý

- Callback trùng → xác nhận kết quả hiện có, không cộng tiền/hủy hold lại.
- Callback success sau booking EXPIRED → payment được ghi nhận; booking NEEDS_ATTENTION và tạo refund/reconciliation, tuyệt đối không tự CONFIRMED.
- Callback tới đúng lúc job hết hạn → một nhánh thắng theo lock/trạng thái, nhánh còn lại giải quyết theo chính sách.
- Provider success nhưng cập nhật DB lỗi → callback retry hoặc reconciliation qua providerReference.

Quy tắc nghiệp vụ / ràng buộc

- Không giữ transaction DB trong thời gian gọi Fake Bank; không lưu số thẻ thật.

Kết quả sau cùng: Payment và booking nhất quán; sự kiện đến trễ/lặp được phục hồi.

Kinh nghiệm kỹ thuật đúc kết: Webhook at-least-once, outbox/reconciliation, nhất quán cuối cùng và tách biệt payment với booking.

UC-11. Tự động hết hạn giữ chỗ

Tác nhân: SCHEDULER

Tiền điều kiện: Có booking PENDING_PAYMENT với expires_at.

Kích hoạt: Job theo lịch kiểm tra booking hết hạn.

Luồng nghiệp vụ chính

Bước 01. Scheduler khởi chạy theo chu kỳ cấu hình; ghi thời điểm bắt đầu và correlation/jobRunId.

Bước 02. Query PostgreSQL lấy batch booking PENDING_PAYMENT có expires_at <= now, dùng khóa/claim tránh xử lý trùng nhiều instance.

Bước 03. Với từng booking, mở transaction ngắn và khóa booking cùng allocation theo thứ tự chuẩn.

Bước 04. Kiểm tra lại trạng thái, hạn giữ và payment mới nhất; bỏ qua nếu đã CONFIRMED hoặc đã EXPIRED.

Bước 05. Nếu chắc chắn hết hạn và chưa có success hợp lệ, booking PENDING_PAYMENT → EXPIRED.

Bước 06. Giải phóng room allocation/hold đúng một lần; ghi booking_status_history và outbox.

Bước 07. Commit từng batch/bản ghi để không giữ lock quá lâu; ghi metric số hết hạn và số lỗi.

Bước 08. Bản ghi xử lý lỗi được retry ở chu kỳ sau với giới hạn và cảnh báo backlog.

Bước 09. Thực hiện kiểm thử chạy song song hai scheduler và callback trả về cùng thời điểm.

Luồng thay thế, ngoại lệ và cách xử lý

- Job crash giữa batch → transaction chưa commit rollback; lần sau xử lý lại.
- Payment vừa success → lock và state check xác định chỉ một nhánh chuyển trạng thái.
- Clock skew → so sánh thời gian server/DB thống nhất; không dựa Redis TTL làm nguồn duy nhất.

Quy tắc nghiệp vụ / ràng buộc

- Job phải recheck trạng thái trong transaction, không tin danh sách đã lấy trước đó.

Kết quả sau cùng: Hold hết hạn không còn chiếm tồn kho; mọi thao tác retry an toàn.

Kinh nghiệm kỹ thuật đúc kết: Batch size, claim row, SKIP LOCKED (nếu chọn), distributed scheduler, lock contention và transaction nhỏ.

UC-12. Khách hủy booking và tính phí hủy

Tác nhân: CUSTOMER/SUPPORT

Tiền điều kiện: Booking thuộc người yêu cầu hoặc SUPPORT có quyền; chính sách snapshot tồn tại.

Kích hoạt: Khách chọn hủy và chấp nhận kết quả phí dự kiến.

Luồng nghiệp vụ chính

Bước 01. Client yêu cầu xem ước tính hủy cho booking; service xác thực quyền sở hữu.

Bước 02. Đọc booking và snapshot chính sách lúc đặt; xác định giờ hủy theo timezone/UTC lưu hệ thống.

Bước 03. Tính phí/tiền dự kiến hoàn dựa vào deadline chính sách, trạng thái booking, payment đã captured.

Bước 04. Trả bản xem trước để khách xác nhận; không dùng chính sách mới do HOST sửa.

Bước 05. Client gửi yêu cầu hủy có idempotencyKey và lý do; server mở transaction khóa booking.

Bước 06. Kiểm tra lại trạng thái và chính sách, chốt cancellation fee và refundableAmount.

Bước 07. Booking chưa thu tiền: chuyển CANCELLED, giải phóng hold, không tạo refund.

Bước 08. Booking đã thu tiền: chuyển CANCELLATION_PENDING hoặc REFUND_PENDING theo state machine, tạo refund REQUESTED khi tiền hoàn > 0.

Bước 09. Ghi cancelled_by/at/reason, lịch sử trạng thái và thông báo; commit.

Bước 10. Các lần gửi lặp chỉ trả cùng kết quả, không tạo refund thứ hai.

Luồng thay thế, ngoại lệ và cách xử lý

- Booking COMPLETED hoặc đã hủy → CANCELLATION_NOT_ALLOWED hoặc idempotent response.
- Quá hạn miễn phí → áp dụng phí theo snapshot, không ước lượng từ giá hiện tại.
- Đang CHECKED_IN → áp dụng quy trình ngoại lệ do SUPPORT/ADMIN xử lý, không hủy tùy tiện.

Quy tắc nghiệp vụ / ràng buộc

- Tổng tiền hoàn <= số tiền thực thu trừ tiền đã hoàn trước đó.

Kết quả sau cùng: Booking/hold/refund đúng chính sách và truy vết được quyết định hủy.

Kinh nghiệm kỹ thuật đúc kết: Phân biệt thao tác hủy đơn với hoàn tiền và quyết định giải phóng tồn; tất cả đều cần chống lặp.

UC-13. Hoàn tiền mô phỏng và đối soát sai lệch

Tác nhân: SYSTEM/SUPPORT

Tiền điều kiện: Có refund REQUESTED hoặc payment đã thành công nhưng booking không thể xác nhận.

Kích hoạt: Phát sinh yêu cầu hoàn tiền hoặc tác vụ đối soát.

Luồng nghiệp vụ chính

Bước 01. Tạo refund với idempotencyKey, paymentId, bookingId, amount và lý do; unique request để chống trùng.

Bước 02. Kiểm tra payment SUCCEEDED và tổng refund đã hoàn/đang xử lý không vượt capturedAmount.

Bước 03. Đưa refund REQUESTED → PROCESSING, commit trạng thái trước khi gọi Fake Bank.

Bước 04. Gửi yêu cầu refund đến Fake Bank với providerReference/correlation nhất quán.

Bước 05. Nhận callback/response; kiểm tra chữ ký, tra providerReference và bắt đầu transaction cập nhật.

Bước 06. Nếu success, refund → SUCCEEDED; cập nhật tổng tiền hoàn đã thực hiện; booking chuyển trạng thái cuối theo quy tắc.

Bước 07. Nếu fail/timeout, lưu mã lỗi và retry theo chính sách; không khởi tạo refund mới cho cùng nghiệp vụ.

Bước 08. Job đối soát so sánh payment/refund nội bộ với Fake Bank, phát hiện callback mất hoặc thành công muộn.

Bước 09. SUPPORT xem danh sách NEEDS_ATTENTION và xử lý có quyền/lý do/audit thay vì sửa DB thủ công.

Luồng thay thế, ngoại lệ và cách xử lý

- Refund callback lặp → không cộng tiền hoàn thêm.
- Provider thành công nhưng DB cập nhật lỗi → giữ cùng providerReference để retry/reconcile.
- Refund tổng vượt captured → REFUND_AMOUNT_EXCEEDED.

Quy tắc nghiệp vụ / ràng buộc

- Tiền thực nhận = tiền đã thu - tiền đã hoàn; không dùng tổng booking như doanh thu thực nhận.

Kết quả sau cùng: Đơn cần hoàn được xử lý hoặc đánh dấu chờ xử lý minh bạch.

Kinh nghiệm kỹ thuật đúc kết: Saga dạng phối hợp nhiều bước, compensation, reconciliation và phép tính tiền nhất quán.

UC-14. Chủ cơ sở xem và xử lý booking

Tác nhân: HOST

Tiền điều kiện: Booking thuộc property có owner_id hiện tại; host có quyền BOOKING_VIEW/UPDATE.

Kích hoạt: HOST mở danh sách đặt phòng hoặc xác nhận yêu cầu đặc biệt.

Luồng nghiệp vụ chính

Bước 01. HOST lọc danh sách booking theo property, ngày, trạng thái; service ép owner scope ngay trong truy vấn.

Bước 02. Repository trả dữ liệu cần thiết, che email/số điện thoại của khách nếu không cần nghiệp vụ.

Bước 03. HOST mở chi tiết và lịch sử trạng thái/ghi chú hỗ trợ được phép.

Bước 04. HOST cập nhật yêu cầu đặc biệt (giờ đến, ghi chú) hoặc phản hồi yêu cầu theo chính sách.

Bước 05. Application service validate quyền, trạng thái hiện tại và trường cho phép sửa.

Bước 06. Ghi trạng thái/ghi chú mới, version và audit actor/reason.

Bước 07. Phát thông báo nội bộ tới khách về cập nhật cần thiết.

Bước 08. Không cho HOST sửa totalAmount, paymentStatus hoặc booking COMPLETED qua API cập nhật chung.

Luồng thay thế, ngoại lệ và cách xử lý

- HOST dùng bookingId của cơ sở khác → FORBIDDEN_SCOPE/404.
- Hai nhân viên HOST sửa cùng booking → VERSION_CONFLICT.
- Yêu cầu thay đổi ảnh hưởng số đêm/số tiền → luồng thay đổi đơn riêng, không sửa trực tiếp.

Quy tắc nghiệp vụ / ràng buộc

- Mọi query host đều có owner_id/property ownership filter.

Kết quả sau cùng: Host xem và xử lý đúng booking của mình, không thể tự thay nghĩa vụ thanh toán.

Kinh nghiệm kỹ thuật đúc kết: Data scope trong repository/service; optimistic locking và audit nghiệp vụ.

UC-15. Check-in, check-out và hoàn tất lưu trú

Tác nhân: HOST

Tiền điều kiện: Booking CONFIRMED, thuộc property được phép xử lý.

Kích hoạt: Khách đến nhận phòng hoặc trả phòng.

Luồng nghiệp vụ chính

Bước 01. HOST tra cứu booking theo mã và thông tin cần thiết đã được cho phép.

Bước 02. Kiểm tra booking CONFIRMED, đúng property và trong cửa sổ check-in theo múi giờ cơ sở.

Bước 03. Xác minh điều kiện nhận phòng và tình trạng RoomUnit; không hiển thị thông tin vượt phạm vi.

Bước 04. HOST thực hiện check-in; transaction khóa booking, recheck status và chuyển CHECKED_IN.

Bước 05. Ghi actual_check_in, actor và lịch sử; nếu thao tác lặp trả kết quả hiện có.

Bước 06. Khi khách trả phòng, HOST mở booking CHECKED_IN; xác nhận điều kiện trả phòng.

Bước 07. Thực hiện check-out: CHECKED_IN → COMPLETED, lưu actual_check_out, cập nhật bản ghi lịch sử.

Bước 08. Phát thông báo hoàn tất và cho phép UC-16 đánh giá một lần.

Bước 09. Báo cáo tình trạng phòng phản ánh đã trả, nhưng giữ lịch sử booking để tra cứu.

Luồng thay thế, ngoại lệ và cách xử lý

- Check-in booking chưa thanh toán/EXPIRED → CHECKIN_NOT_ALLOWED.
- Check-out khi chưa check-in → chỉ SUPPORT/ADMIN qua quy trình ngoại lệ có lý do.
- Trùng request hoặc quầy lễ tân thao tác đồng thời → state check/locking chống cập nhật hai lần.

Quy tắc nghiệp vụ / ràng buộc

- Hoàn tất lưu trú không đồng nghĩa được phép thay tổng tiền đã xác nhận.

Kết quả sau cùng: Booking COMPLETED, lịch sử lưu trú đầy đủ.

Kinh nghiệm kỹ thuật đúc kết: State machine không thay thế kiểm tra quyền và mốc thời gian; lưu business date vs timestamp chính xác.

UC-16. Đánh giá và xử lý phản hồi

Tác nhân: CUSTOMER/SUPPORT

Tiền điều kiện: Booking COMPLETED, customer sở hữu booking và chưa từng đánh giá.

Kích hoạt: Khách gửi nhận xét sau lưu trú.

Luồng nghiệp vụ chính

Bước 01. Khách mở booking đã hoàn tất; hệ thống kiểm tra điều kiện đánh giá.

Bước 02. Client gửi nội dung nhận xét, mức đánh giá chất lượng theo thang được hệ thống quy định và ảnh nếu hỗ trợ.

Bước 03. Server lọc nội dung không hợp lệ, giới hạn độ dài, kiểm tra room/property trùng với booking.

Bước 04. Tạo review gắn bookingId, propertyId, customerId; unique bookingId ngăn đánh giá trùng.

Bước 05. Đặt trạng thái PUBLISHED hoặc PENDING_MODERATION theo chính sách kiểm duyệt.

Bước 06. Cập nhật aggregate hiển thị/tác vụ refresh cache mà không làm hỏng giao dịch tạo review.

Bước 07. HOST được xem và có thể phản hồi theo chức năng riêng, không sửa nội dung review gốc.

Bước 08. SUPPORT nhận báo cáo đánh giá vi phạm, xem bằng chứng và quyết định ẩn/khôi phục có audit.

Luồng thay thế, ngoại lệ và cách xử lý

- Booking chưa COMPLETED → REVIEW_NOT_ELIGIBLE.
- Gửi lại cùng booking → REVIEW_ALREADY_EXISTS.
- Báo cáo sai phạm → trạng thái moderation thay đổi nhưng dữ liệu gốc vẫn lưu để truy vết.

Quy tắc nghiệp vụ / ràng buộc

- Phản hồi chất lượng lưu trú là dữ liệu nghiệp vụ và phải gắn với booking đã hoàn tất.

Kết quả sau cùng: Review hợp lệ và kiểm duyệt truy vết được.

Kinh nghiệm kỹ thuật đúc kết: Unique constraint theo booking, policy moderation, phân quyền chỉnh sửa và nhất quán eventual cho chỉ số tổng hợp.

UC-17. Thông báo nội bộ theo sự kiện nghiệp vụ

Tác nhân: SYSTEM

Tiền điều kiện: Có thay đổi booking/payment/cancellation/check-in cần thông báo.

Kích hoạt: Sự kiện nghiệp vụ đã commit.

Luồng nghiệp vụ chính

Bước 01. Use case phát sinh bản ghi outbox cùng transaction khi booking/thanh toán thay đổi.

Bước 02. Worker đọc outbox theo batch, claim bản ghi và ánh xạ event sang mẫu thông báo phù hợp.

Bước 03. Xác định recipient dựa trên quan hệ thật từ DB, không tin userId do event ngoài gửi.

Bước 04. Tạo notifications với loại sự kiện, tiêu đề, nội dung rút gọn, object reference và thời điểm.

Bước 05. Đánh dấu outbox đã xử lý; nếu lỗi thì tăng retryCount và backoff, không mất sự kiện.

Bước 06. Client mở trang thông báo; API lọc recipient_id bằng principal.

Bước 07. Người dùng đánh dấu đã đọc; cập nhật read_at idempotent.

Bước 08. Thông báo không chứa bí mật, ảnh private URL dài hạn hay dữ liệu thanh toán nhạy cảm.

Luồng thay thế, ngoại lệ và cách xử lý

- Worker restart khi đã tạo notification nhưng chưa mark outbox → unique eventId + recipient tránh trùng.
- Event chứa tài nguyên bị xóa/ẩn → thông báo vẫn giữ metadata an toàn hoặc ngừng liên kết theo quyền.
- Người dùng khác sửa notificationId → chặn bằng recipient scope.

Quy tắc nghiệp vụ / ràng buộc

- Outbox phù hợp cho side effect sau commit; không gửi thông báo thành công khi transaction chính rollback.

Kết quả sau cùng: Mỗi sự kiện tạo tối đa một thông báo hiệu lực cho mỗi người nhận.

Kinh nghiệm kỹ thuật đúc kết: Đảm bảo giao hàng ít nhất một lần và xử lý idempotent phía consumer.

UC-18. Báo cáo đặt phòng, doanh thu và tình trạng phòng

Tác nhân: HOST/ADMIN

Tiền điều kiện: Có dữ liệu booking/payment/refund và quyền đọc theo property.

Kích hoạt: Người quản lý chọn kỳ báo cáo.

Luồng nghiệp vụ chính

Bước 01. Nhận propertyId, từ ngày/đến ngày, trạng thái và loại thống kê; kiểm tra phạm vi quyền.

Bước 02. Chuẩn hóa thời gian theo timezone property, thống nhất cách tính kỳ theo booking_date hoặc stay_date.

Bước 03. Query booking count theo trạng thái, số đêm đã bán, tỷ lệ lấp đầy trong kỳ, số lượt hủy.

Bước 04. Tách các thước đo: booking value; captured payment; succeeded refunds; net collected = captured - refunded.

Bước 05. Loại các booking đã hủy khỏi một số chỉ số lưu trú theo định nghĩa và công bố công thức.

Bước 06. Dùng projection/aggregate SQL và index/partition nếu dữ liệu lớn; không tải mọi entity để tính tại Java.

Bước 07. Trả dữ liệu tổng hợp phân quyền theo property; cache số liệu nếu cho phép độ trễ và hiển thị asOf.

Bước 08. Hỗ trợ đối chiếu chi tiết từng booking/payment khi có lệch số.

Bước 09. Ghi thời gian chạy, query plan và metric độ trễ của báo cáo.

Luồng thay thế, ngoại lệ và cách xử lý

- HOST xem property khác → FORBIDDEN_SCOPE.
- Callback hoàn tiền muộn → số liệu kỳ có thể thay đổi; báo cáo gắn asOf.
- Dữ liệu không nhất quán → hiển thị danh sách đối soát, không tự âm thầm cân bằng bằng công thức sai.

Quy tắc nghiệp vụ / ràng buộc

- Định nghĩa metrics nghiệp vụ phải nhất quán giữa màn hình và xuất báo cáo.

Kết quả sau cùng: Báo cáo minh bạch về phương pháp tính, quyền và thời điểm dữ liệu.

Kinh nghiệm kỹ thuật đúc kết: Aggregate SQL, indexing, projection, báo cáo eventual-consistent và trách nhiệm định nghĩa chỉ số.

UC-19. Audit thao tác nhạy cảm và hỗ trợ tra cứu sự cố

Tác nhân: SUPPORT/ADMIN

Tiền điều kiện: Actor có quyền tra cứu audit; cấu hình retention và dữ liệu nhạy cảm.

Kích hoạt: Người có quyền tra một thao tác hoặc hệ thống ghi lại thay đổi.

Luồng nghiệp vụ chính

Bước 01. Ở use case nhạy cảm, tạo audit event với actorId, action, entityType, entityId, requestId, from/to state và lý do.

Bước 02. Che dữ liệu nhạy cảm trước khi lưu; không lưu password, access token, refresh token hay signed URL.

Bước 03. Ghi audit đồng bộ trong transaction cho thay đổi trạng thái cốt lõi hoặc dùng outbox nhất quán theo chính sách.

Bước 04. SUPPORT/ADMIN gửi bộ lọc theo actor, entity, khoảng thời gian, correlationId; server kiểm tra quyền truy cập.

Bước 05. Truy vấn theo index; phân trang và trả projection giới hạn trường.

Bước 06. Cho phép truy vết booking ↔ payment ↔ refund ↔ job bằng requestId/eventId.

Bước 07. Khi tra cứu, ghi thêm nhật ký truy cập audit nếu dữ liệu nhạy cảm.

Bước 08. Bảo đảm audit không được sửa/xóa bởi API nghiệp vụ thông thường; quản lý retention có kiểm soát.

Luồng thay thế, ngoại lệ và cách xử lý

- Transaction chính rollback nhưng audit kiểu REQUIRES_NEW đã commit → phân biệt attempt với success rõ ràng.
- Không đủ quyền đọc audit → 403.

- Log thiếu correlation ID → có thể tra theo entityId và timestamp, nhưng phải bổ sung instrumentation.

Quy tắc nghiệp vụ / ràng buộc

- Audit thành công phải phản ánh commit thực tế; event "đã thử" khác event "thành công".

Kết quả sau cùng: Có chuỗi sự kiện đủ điều tra và đối soát; dữ liệu riêng tư được che.

Kinh nghiệm kỹ thuật đúc kết: AOP cho cross-cutting concerns, Spring proxy, transaction propagation, security audit và traceability.

UC-20. Quản trị tài khoản, quyền và danh mục hệ thống

Tác nhân: ADMIN

Tiền điều kiện: ADMIN có quyền tương ứng, tài khoản đang ACTIVE và cơ chế audit hoạt động.

Kích hoạt: Quản trị thay đổi cấu hình/quyền hoặc khóa tài khoản.

Luồng nghiệp vụ chính

Bước 01. ADMIN xác thực rồi mở danh sách users, roles, permissions và danh mục amenities.

Bước 02. Server giới hạn truy vấn, lọc và phân trang; che trường nhạy cảm của tài khoản.

Bước 03. ADMIN chọn khóa/mở khóa user, cấp/thu quyền hoặc cập nhật danh mục.

Bước 04. Application service kiểm tra quyền cụ thể, validation, policy phân quyền và hạn chế tự thu quyền quản trị cuối cùng nếu áp dụng.

Bước 05. Cập nhật bằng transaction, ghi phiên bản cấu hình và audit actor/reason.

Bước 06. Khi khóa user, đổi tokenVersion/denylist refresh token và áp dụng chính sách đối với access token chưa hết hạn.

Bước 07. Khi đổi danh mục công khai, phát invalidation cache thích hợp.

Bước 08. Trả kết quả và cảnh báo ảnh hưởng nếu user đang sở hữu property/booking.

Bước 09. Kiểm thử user cố sửa role bằng API hồ sơ thông thường phải bị chặn.

Luồng thay thế, ngoại lệ và cách xử lý

- Role/permission đang sử dụng → không xóa cứng làm đứt lịch sử.
- ADMIN không có permission cụ thể → 403 dù có role tên ADMIN nếu dùng permission granular.
- Khóa HOST có booking đang hoạt động → vẫn giữ dữ liệu và quy trình hỗ trợ xử lý booking.

Quy tắc nghiệp vụ / ràng buộc

- Cấp quyền cần tách API, DTO và audit khỏi sửa hồ sơ thông thường.

Kết quả sau cùng: Quản trị có truy vết và không làm hỏng dữ liệu giao dịch lịch sử.

Kinh nghiệm kỹ thuật đúc kết: Least privilege, phân tách quyền theo chức năng và phạm vi, token revocation, kiểm soát cấu hình.

3.1. Bổ sung các use case báo cáo, trao đổi, email và Fake Bank

Các use case UC-21 đến UC-36 bổ sung chi tiết cho các phân hệ hiện có. Mọi báo cáo phải nêu rõ ngữ nghĩa thời gian, nguồn số liệu, phạm vi dữ liệu và độ trễ cập nhật. Giao dịch ngân hàng giả lập được tách rời ranh giới transaction của hệ thống đặt phòng.

UC-21. Thống kê dashboard tổng quan theo phạm vi dữ liệu

Tác nhân: HOST, ADMIN.

Tiền điều kiện: HOST đã đăng nhập và được gắn owner scope; hệ thống có booking/payment/refund đã ghi nhận.

Luồng nghiệp vụ chính

Bước 01. Nhận bộ lọc gồm khoảng thời gian, timezone property, tập property thuộc quyền, loại phòng, trạng thái booking và khoảng giá.

Bước 02. Xác thực quyền và chuẩn hóa ngày theo múi giờ từng cơ sở; không dùng ngày UTC thô để nhóm ngày lưu trú.

Bước 03. Gán metric riêng: số booking tạo, booking được xác nhận, đêm bán, tổng tiền đã thu, tiền đã hoàn, tổng tiền thực thu, số phòng đang mở bán.

Bước 04. Đọc cache Redis theo key chứa tenant/owner scope, filters chuẩn hóa, ngày và phiên bản dữ liệu; cache hit trả cả thời điểm cập nhật.

Bước 05. Cache miss sử dụng SQL aggregate trên dữ liệu gốc hoặc bảng tổng hợp; áp dụng cùng phạm vi dữ liệu ở mọi nhánh CTE.

Bước 06. Tính delta so với kỳ trước có cùng độ dài và múi giờ nếu người dùng yêu cầu; tránh so sánh tháng 30 ngày với 31 ngày mà không nêu quy ước.

Bước 07. Đóng gói số liệu theo currency và trạng thái, ghi version dữ liệu và TTL; trả biểu đồ theo ngày/tuần/tháng.

Bước 08. Ghi metric query_time/cache_hit, trả thông tin asOf và sourceType (LIVE/AGGREGATE/CACHE).

Luồng thay thế, lỗi và ngoại lệ

– Không có quyền một property: bỏ property khỏi kết quả hoặc trả FORBIDDEN_SCOPE nếu yêu cầu trực tiếp; không trả số liệu cộng gộp vượt quyền.

– Tập lọc quá lớn: giới hạn khoảng ngày, số property và số nhóm; yêu cầu export bất đồng bộ thay vì truy vấn vô hạn.

– Cache lỗi: fallback SQL khi đủ tài nguyên, không biến số liệu lỗi thời thành dữ liệu xác nhận theo thời gian thực.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Không cộng booking đã hủy vào doanh thu thực nhận; phân biệt gross booking, captured, refund succeeded và net captured.

– Không tính trung bình đơn giản của phần trăm từng property khi tổng hợp; dùng tử số/mẫu số tương ứng.

– Test quyền dữ liệu ở mức SQL, Redis key isolation và sai lệch timezone cuối tháng.

Yêu cầu kỹ thuật và minh chứng

PostgreSQL CTE/filtered aggregates + Redis cache-aside; EXPLAIN (ANALYZE, BUFFERS) cho bộ lọc phổ biến; metric số lượng series và DB reads.

UC-22. Thống kê đặt phòng đa chiều và so sánh kỳ

Tác nhân: HOST, ADMIN.

Tiền điều kiện: Có các booking qua nhiều cơ sở, phòng và kênh; bộ lọc hợp lệ.

Luồng nghiệp vụ chính

Bước 01. Chọn khoảng thời gian theo created_at, check_in hay check_out (bắt buộc chọn một ngữ nghĩa thời gian).

Bước 02. Chọn trạng thái, property, roomType, khoảng tiền, số đêm, số khách, nguồn đặt phòng và nhóm khách quay lại/mới.

Bước 03. Xác minh owner scope; normalize filters và phân trang/sort whitelist trên các cột cho phép.

Bước 04. Tạo truy vấn dynamic qua Specification/Criteria hoặc SQL CTE; join booking_nights chỉ khi cần metric theo đêm, tránh nhân bản booking.

Bước 05. Nhóm theo ngày/tuần/tháng, property, roomType, trạng thái hoặc tổ hợp tối đa được cấu hình.

Bước 06. Tính count distinct booking và tổng số đêm chính xác; bộ lọc trạng thái áp dụng trước aggregate.

Bước 07. Đọc kỳ so sánh cùng quy tắc; hiển thị số tuyệt đối, mức tăng/giảm và mẫu số của tỷ lệ.

Bước 08. Trả breakdown, totalRows, dữ liệu từ cache hay DB và asOf; hỗ trợ drill-down vào danh sách booking có phân trang.

Luồng thay thế, lỗi và ngoại lệ

– Không dùng COUNT(*) sau join booking_nights để đếm booking vì sẽ bị nhân lên theo đêm.

– Filter thời gian không tương thích với requested grouping: trả VALIDATION_ERROR, giải thích cách chọn.

– Nhóm quá nhiều chiều/tổ hợp: dùng materialized aggregate hoặc background export có giới hạn.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Các nhóm cộng lại phải khớp metric tổng khi cùng nguồn và filter; timezone từng property rõ ràng.

– Có test 1 booking 3 đêm chỉ được đếm một booking và 3 room-nights.

– Index tối ưu thứ tự scope/time/status theo selectivity và dữ liệu EXPLAIN thực tế.

Yêu cầu kỹ thuật và minh chứng

Dynamic SQL có bind params, composite index (owner/property/time/status), tránh OFFSET sâu bằng keyset cho drill-down khi phù hợp.

UC-23. Thống kê doanh thu, dòng tiền và hoàn tiền nhiều điều kiện

Tác nhân: HOST, ADMIN.

Tiền điều kiện: Booking và bảng payment/refund có các mốc thời gian được lưu đầy đủ.

Luồng nghiệp vụ chính

Bước 01. Người sử dụng chọn một trục thời gian: ngày đặt, ngày captured, ngày hoàn tiền hoặc ngày lưu trú; hệ thống hiển thị định nghĩa trục đang dùng.

Bước 02. Nhận filter property, loại phòng, loại giá, trạng thái, phương thức thanh toán giả lập, khoản tiền và khoảng ngày.

Bước 03. Đọc payment SUCCEEDED/CAPTURED theo thời điểm thu; chỉ đọc refund SUCCEEDED theo thời điểm hoàn; loại giao dịch pending/failed ra khỏi net cash.

Bước 04. Tính gross bookings, captured amount, refunded amount, net cash, số booking có thanh toán, tỷ lệ refund và số giao dịch chờ đối soát.

Bước 05. Để tính doanh thu lưu trú theo đêm, phân bổ theo booking_nights snapshot, tách riêng với dòng tiền thực thu.

Bước 06. Nhóm theo property/roomType/ngày; dùng SUM numeric/decimal, currency không trộn lẫn.

Bước 07. Khi booking hủy nhưng payment đã thu, vẫn ghi nhận dòng tiền và refund ở kỳ tương ứng, không xóa lịch sử giao dịch.

Bước 08. Báo cáo có cutoff/asOf và quy tắc xử lý giao dịch hoàn tiền đến muộn.

Bước 09. Cache dữ liệu tổng hợp theo version hoặc invalidate khi payment/refund trạng thái cuối được commit.

Luồng thay thế, lỗi và ngoại lệ

– Refund pending không trừ vào tiền đã hoàn; hiển thị riêng dự kiến hoàn.

– Payment success ở Fake Bank nhưng booking chưa đồng bộ: đưa vào diện reconciliation, không tự nhận booking confirmed.

– Đa tiền tệ (nếu triển khai) phải tách theo currency, không cộng trực tiếp.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Define rõ doanh thu đặt phòng khác doanh thu ghi nhận theo lưu trú và dòng tiền.

– Không join trực tiếp payments với refunds và booking_nights rồi SUM vì tạo Cartesian multiplier.

– Test payment partial/refund partial, callback trùng, hủy xuyên kỳ, trạng thái reconciliation.

Yêu cầu kỹ thuật và minh chứng

Ưu tiên aggregate riêng payment/refund/booked_nights rồi join kết quả theo dimension; SQL window/CTE; bảng daily_financial_fact khi dữ liệu lớn.

UC-24. Thống kê công suất phòng, tồn kho và giá theo ngày

Tác nhân: HOST, ADMIN.

Tiền điều kiện: Có room unit, lịch đóng bán, booking và lịch giá theo ngày.

Luồng nghiệp vụ chính

Bước 01. Người sử dụng chọn property, roomType, khoảng ngày, ngày trong tuần, tình trạng phòng, sức chứa và dải giá.

Bước 02. Xác định số phòng có thể bán theo từng stay_date: loại phòng inactive/maintenance và ngày bị khóa bán.

Bước 03. Xác định số room-night đã chiếm từ booking CONFIRMED/CHECKED_IN/COMPLETED có overlap; held có thể được báo cáo riêng.

Bước 04. Tính số room-night khả dụng và công suất = occupied room-nights / sellable room-nights khi mẫu số > 0.

Bước 05. Tính ADR = doanh thu phòng phân bổ / occupied room-nights, RevPAR = doanh thu phòng phân bổ / sellable room-nights, ghi rõ thuế/phí có tính hay không.

Bước 06. Aggregate theo từng ngày rồi theo tuần/tháng/property; tính tỷ lệ bằng tổng tử số chia tổng mẫu số.

Bước 07. So sánh công suất với lịch giá; phân nhóm cuối tuần/giữa tuần hoặc theo dải giá.

Bước 08. Trả heatmap công suất kèm danh sách ngày cao tải; cache ngắn và invalidate sau thay đổi lịch, booking, giá.

Luồng thay thế, lỗi và ngoại lệ

– Mẫu số bằng 0: trả null hoặc N/A, không hiển thị 0% sai nghĩa.

– Booking checkOut cùng ngày checkIn của booking khác không overlap vì [checkIn, checkOut).

– Room maintenance phát sinh trong khoảng đã bán phải được phát hiện để xử lý ngoại lệ.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Phải thống nhất booking status nào chiếm tồn và metric occupied vs held.

– Giữ snapshot giá lịch sử khi đổi rate hiện tại.

– Test ngày nhuận, timezone, room đóng bán một số ngày và booking nhiều đêm.

Yêu cầu kỹ thuật và minh chứng

Date spine (generate_series), anti-join inventory, composite index (room_id, stay_date), daily occupancy fact; Redis TTL + version theo property/ngày.

UC-25. Thống kê tỷ lệ hủy, lỗi Fake Bank và phễu đặt phòng

Tác nhân: HOST, ADMIN, SUPPORT được phân quyền.

Tiền điều kiện: Có nhật ký business event theo bước và trạng thái ổn định.

Luồng nghiệp vụ chính

Bước 01. Chọn thời gian, property, loại phòng, nhóm khách, nguồn traffic, payment fail code và nguyên nhân hủy.

Bước 02. Phân biệt bước funnel: search được hiển thị, quote đã cấp, hold tạo, payment initiated, payment captured, booking confirmed, check-in hoàn tất.

Bước 03. Dùng event_id/request_id để giảm đếm trùng do retry; mỗi unique funnel session chỉ đếm một lần tại một bước theo quy định.

Bước 04. Tính conversion mỗi bước và toàn luồng; metric cancel count theo reason, trước/sau check-in và khoảng thời gian đến checkIn.

Bước 05. Nhóm failure Fake Bank theo INSUFFICIENT_FUNDS, DECLINED, TIMEOUT, CALLBACK_LATE, DUPLICATE, DB_COMMIT_ERROR; không coi timeout mặc định thất bại.

Bước 06. Phân tích thời gian từ giữ chỗ đến thanh toán và từ thanh toán đến xác nhận; dùng percentile nếu đủ dữ liệu.

Bước 07. Cho drill-down theo payment transaction/reference và booking với mask dữ liệu riêng tư.

Bước 08. Trả biểu đồ và thống kê số lượng nền mẫu, tránh kết luận từ tỷ lệ với mẫu quá nhỏ.

Luồng thay thế, lỗi và ngoại lệ

– Mất event analytics: không suy diễn funnel hoàn chỉnh từ riêng bảng booking; hiển thị coverage.

– Callback lặp: không tạo thêm captured conversion; dựa idempotent event record.

– SUPPORT chỉ xem phạm vi xử lý được chỉ định và dữ liệu đã che.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Phân biệt event-time với ingestion-time; thống nhất attribution window.

– Không chứa email, số điện thoại, message hay PII trong metric label.

– Test hoạt động khi event đến muộn hoặc bị gửi lại.

Yêu cầu kỹ thuật và minh chứng

Event log append-only với unique event ID; partition/index event_time + property; CTE/window functions và Redis cache cho mẫu tổng hợp.

UC-26. Truy vấn thống kê động, xuất dữ liệu và cache Redis

Tác nhân: HOST, ADMIN.

Tiền điều kiện: Các KPI được định nghĩa, quyền data scope đã thống nhất.

Luồng nghiệp vụ chính

Bước 01. Nhận request JSON gồm metric whitelist, filters, groupBy tối đa 3 chiều, sort, dateGranularity, giới hạn kết quả.

Bước 02. Validate tổ hợp metric/dimension được phép, giới hạn range theo loại báo cáo, chuẩn hóa timezone và ngăn query injection.

Bước 03. Tạo canonical key từ tenantId, principal scope fingerprint, metrics, filters, groupBy, sort, granularity, schemaVersion.

Bước 04. Tra Redis: HIT trả kết quả kèm asOf, TTL remaining, cache version; MISS xây query tham số hóa, có statement_timeout và row limit.

Bước 05. Đọc explain plan trên môi trường kiểm thử; phân tích scan, sort, hash aggregate, index, shared buffers và lock duration.

Bước 06. Với điều kiện phức tạp/lượng lớn chuyển sang bảng fact theo ngày (daily_property_kpi, daily_room_kpi, daily_finance_kpi) cập nhật incremental.

Bước 07. Sau commit booking/payment/refund/room update, phát invalidation event có tính idempotent; áp dụng cache version để tránh ghi lại cache cũ sau invalidation.

Bước 08. Nếu người dùng yêu cầu export rất lớn, tạo report_job trạng thái QUEUED/RUNNING/COMPLETED/FAILED; xuất CSV trong object storage an toàn và URL có hạn.

Bước 09. Ghi audit ai trích xuất dữ liệu, số bản ghi, bộ lọc; bảo vệ CSV formula injection, giới hạn tuổi file và thu hồi URL.

Bước 10. Kiểm tra đối chiếu tổng hợp cache/materialized với SQL gốc trên cùng cutoff và snapshot.

Luồng thay thế, lỗi và ngoại lệ

– Redis unavailable: fallback DB có rate limit/circuit breaker, hoặc trả tạm thời không khả dụng cho báo cáo quá nặng.

– Trùng report job: cùng requestId/hash trả job cũ, không khởi chạy hàng loạt.

– Đổi owner scope giữa lúc cache còn hạn: key/fingerprint phải thay đổi hoặc vô hiệu hóa.

– Bảng aggregate lag: luôn trả asOf và lagSeconds thay vì tuyên bố real time.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Cache chỉ tăng tốc, PostgreSQL là nguồn tính đúng đắn; không cache chung giữa tenants.

– Chứng minh cải thiện bằng số đo EXPLAIN, p95, rows scanned và cache hit ratio trên dữ liệu mẫu đủ lớn.

– Bảo vệ concurrency truy vấn nặng và giới hạn số request active.

Yêu cầu kỹ thuật và minh chứng

PostgreSQL partial/composite index, partitions chỉ khi cần, pre-aggregation/upsert theo event id, Redis cache-aside+TTL+invalidation; kịch bản cache stampede.

UC-27. Tạo hội thoại giữa khách và chủ cơ sở

Tác nhân: CUSTOMER, HOST.

Tiền điều kiện: Khách đã đăng nhập; property PUBLISHED hoặc có booking tương ứng.

Luồng nghiệp vụ chính

Bước 01. Khách chọn Liên hệ chủ cơ sở từ trang property hoặc booking để trao đổi yêu cầu trước khi đặt/chăm sóc lưu trú.

Bước 02. Hệ thống xác minh property hiển thị công khai hoặc booking thuộc CUSTOMER; không tin hostId truyền từ client.

Bước 03. Kiểm tra chặn spam, số cuộc hội thoại đang mở và chính sách cho phép liên hệ trước khi đặt.

Bước 04. Tạo conversation gồm property_id, customer_id, host_id theo owner thực tế, optional booking_id, type INQUIRY/BOOKING_SUPPORT, status OPEN.

Bước 05. Với cùng booking và conversation type có thể tái sử dụng conversation đang hoạt động theo unique business key.

Bước 06. Ghi tin nhắn đầu tiên trong transaction DB; tạo notification/outbox để báo HOST.

Bước 07. Trả conversationId, quyền tham gia, tin nhắn đầu và thời điểm đọc cuối; giao diện hiển thị theo cursor pagination.

Bước 08. HOST truy cập inbox chỉ nhận hội thoại thuộc property đang sở hữu; thông tin khách nhạy cảm không tự công khai.

Luồng thay thế, lỗi và ngoại lệ

– Property chuyển owner hoặc archived: xác định chủ có quyền xem lịch sử và người kế nhiệm; không lộ hội thoại cho chủ cũ.

– Khách bị khóa hoặc spam: chặn tạo mới theo policy; yêu cầu hỗ trợ qua kênh được phép.

– Phát hai request khởi tạo song song: DB unique + upsert/idempotency không tạo hội thoại trùng.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Cho phép INQUIRY trước booking nhưng không chia sẻ thông tin thanh toán hoặc tài liệu riêng tư.

– Mọi conversation có owner scope, participant list, created_at, closed_at và lịch sử.

– Không dùng Redis Pub/Sub làm nơi lưu lịch sử tin nhắn.

Yêu cầu kỹ thuật và minh chứng

PostgreSQL conversation/message append-only, unique active conversation key; Redis rate limit/unread count cache, outbox thông báo.

UC-28. Nhắn tin, phản hồi phản ánh và chuyển xử lý hỗ trợ

Tác nhân: CUSTOMER, HOST, SUPPORT.

Tiền điều kiện: Có conversation hợp lệ; người gửi là participant hoặc SUPPORT được chỉ định.

Luồng nghiệp vụ chính

Bước 01. Người gửi nhập nội dung, chọn loại NORMAL/COMPLAINT/CHECKIN_REQUEST, có thể kèm assetId hợp lệ.

Bước 02. Validate độ dài, nội dung độc hại, file ảnh và quyền object MinIO; loại bỏ HTML không an toàn.

Bước 03. Gửi message với clientMessageId do client tạo; unique(conversation_id,sender_id,client_message_id) chống gửi trùng khi mạng retry.

Bước 04. Tạo message trạng thái SENT, created_at UTC, correlationId và sequence có thứ tự trong conversation.

Bước 05. Đẩy sự kiện outbox; tùy chọn SSE/WebSocket có thể cập nhật gần thời gian thực, nhưng message DB luôn là nguồn sự thật.

Bước 06. Người nhận mở conversation; cập nhật last_read_seq idempotent, tính unread = seq mới nhất − seq đã đọc.

Bước 07. Khách đánh dấu phản ánh: tạo complaint ticket với loại vấn đề, mức độ, deadline phản hồi theo cấu hình.

Bước 08. HOST trả lời trong cùng thread, ghi nhận người xử lý; ticket RESOLVED chỉ khi có lý do/biên bản hoặc khách xác nhận theo policy.

Bước 09. Khi quá hạn hoặc có tranh chấp, SUPPORT được phân công xem thread đủ phạm vi, xử lý và audit.

Bước 10. Cho lọc inbox theo chưa đọc, chưa trả lời, có khiếu nại, booking sắp nhận phòng.

Luồng thay thế, lỗi và ngoại lệ

– Message bị gửi lại: trả bản ghi cũ; cùng clientMessageId nhưng nội dung khác trả IDEMPOTENCY_CONFLICT.

– HOST mất quyền sở hữu property: chặn truy cập cho những thread không còn thuộc phạm vi được giao.

– SSE/WebSocket rớt: client đồng bộ từ cursor và không mất tin.

– Không thể gửi mail/thông báo: message vẫn được commit; outbox retry độc lập.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Tin nhắn không được sửa/xóa cứng để xóa dấu vết phản ánh; nếu cần chỉ redaction với audit.

– Không cho HOST tự đánh dấu phản ánh của khách là đã giải quyết nếu quy trình cần xác nhận.

– Không ghi nội dung nhắn tin vào metric label, search log hay URL.

Yêu cầu kỹ thuật và minh chứng

Optimistic sequence/version cho read receipt; indexed cursor pagination, outbox, Redis rate limit/thống kê unread.

UC-29. Đề nghị giờ check-in và xác nhận qua hội thoại

Tác nhân: CUSTOMER, HOST.

Tiền điều kiện: Booking CONFIRMED, trong thời hạn chính sách và đúng property.

Luồng nghiệp vụ chính

Bước 01. Khách mở booking chọn Yêu cầu check-in sớm/muộn hoặc báo đến trễ; nhập khung giờ mong muốn và lời nhắn.

Bước 02. Hệ thống kiểm tra booking owner, ngày check-in theo timezone property, booking status và hạn gửi yêu cầu.

Bước 03. Tạo checkin_request gắn booking_id, proposed_at, note, status PENDING, version và message conversation liên quan.

Bước 04. HOST nhận inbox/notification; chọn APPROVED, REJECTED, COUNTER_PROPOSED và trả lời kèm thời điểm được chấp nhận.

Bước 05. Khi khách đồng ý đề xuất khác, hệ thống lưu agreed_checkin_at; chỉ là giờ hẹn, không tự đổi booking thành CHECKED_IN.

Bước 06. Tới ngày nhận phòng, HOST xác nhận khách thực đến bằng hành động check-in riêng, kiểm tra quyền và trạng thái.

Bước 07. Sau check-in ghi checked_in_at, actor, phương thức xác minh, cập nhật thread và gửi email xác nhận nếu cấu hình bật.

Bước 08. Nếu khách không đến trong hạn quy định, HOST gửi nhắc; sau cutoff chuyển xử lý NO_SHOW qua quy trình riêng có audit.

Bước 09. Đóng request và ghi lịch sử trạng thái; email nhắc check-in sử dụng agreed_checkin_at nếu có, nếu không dùng thời gian mặc định.

Luồng thay thế, lỗi và ngoại lệ

– Hai HOST đồng thời phản hồi: optimistic locking chỉ một thay đổi cuối hợp lệ, yêu cầu kia nhận CONFLICT.

– Booking bị hủy trong lúc còn yêu cầu: tự đóng request và ngừng mail nhắc.

– Khách xin giờ ngoài thời gian chấp nhận: từ chối với lý do/đề xuất thay thế.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Phản hồi trong chat không đồng nghĩa check-in thực tế; state booking là nguồn sự thật.

– Không cho check-in booking chưa thanh toán nếu policy bắt buộc CONFIRMED.

– Nhắc mail chỉ một lần cho mỗi booking + loại nhắc + mốc cấu hình.

Yêu cầu kỹ thuật và minh chứng

State machine checkin_request; transaction kiểm tra booking/version; outbox email và cron scheduler.

UC-30. Cấu hình chính sách mật khẩu và tham số nghiệp vụ

Tác nhân: ADMIN.

Tiền điều kiện: Quản trị viên có quyền SYSTEM_CONFIG_WRITE và xác thực tăng cường với thay đổi nhạy cảm.

Luồng nghiệp vụ chính

Bước 01. ADMIN mở cấu hình gồm password.minLength, password.maxLength, yêu cầu ký tự, max failed login, token TTL, emailReminder.beforeCheckin, timezone, retry policy.

Bước 02. API cung cấp giá trị, kiểu dữ liệu, phạm vi GLOBAL/TENANT/PROPERTY, giới hạn min/max và version.

Bước 03. ADMIN nhập độ dài tối thiểu/tối đa (ví dụ 10/64, chỉ là giá trị cấu hình mẫu), thời gian gửi nhắc (ví dụ 48 giờ) và thời gian yên lặng.

Bước 04. Backend validate minLength <= maxLength, không cho cấu hình vô nghĩa/không an toàn, ví dụ độ dài tối thiểu dưới mức floor hệ thống.

Bước 05. Tạo bản ghi cấu hình version mới, effective_at, updated_by và ghi audit before/after không chứa secret.

Bước 06. Invalidate cache cấu hình có version; các instance tải version mới qua polling/pubsub có cơ chế fallback DB.

Bước 07. Khi đăng ký/đổi mật khẩu tiếp theo, kiểm tra policy hiệu lực tại thời điểm xử lý; không cưỡng bức đổi ngay mật khẩu cũ nếu không có migration policy.

Bước 08. Job nhắc check-in đọc cấu hình hiệu lực theo tenant/property, chốt reminder_due_at cho từng booking hoặc tạo schedule dựa trên policy rõ ràng.

Bước 09. Trả API cấu hình công khai tối thiểu cho form đăng ký: minLength, maxLength, policy summary; không lộ secret cấu hình.

Luồng thay thế, lỗi và ngoại lệ

– ADMIN tăng minLength từ 10 lên 14: tài khoản cũ vẫn đăng nhập trừ khi chính sách ép reset.

– Đổi thời gian gửi nhắc khi job đã lên lịch: áp dụng quy tắc EFFECTIVE_FOR_UNSENT hoặc SNAPSHOT_AT_BOOKING do hệ thống chọn.

– Config cache lệch giữa instance: sử dụng version/idempotency và audit để phát hiện.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Độ dài phải đo theo Unicode code points hay độ dài byte và thống nhất backend/frontend; tránh truncation bí mật.

– Password hash bằng BCrypt/Argon2 theo capability, không lưu reversible secret.

– Biến cấu hình không phải mọi thay đổi đều được áp dụng tức thì với job đang chạy.

Yêu cầu kỹ thuật và minh chứng

config_definitions/config_values/config_history; optimistic version, Redis config cache TTL ngắn/invalidate theo version.

UC-31. Quản lý mẫu email có biến thay thế và phiên bản

Tác nhân: ADMIN.

Tiền điều kiện: SMTP/fake SMTP đã cấu hình; danh mục event email được định nghĩa.

Luồng nghiệp vụ chính

Bước 01. ADMIN tạo template theo event ACCOUNT_CREATED, EMAIL_VERIFICATION, BOOKING_CONFIRMED, CHECKIN_REMINDER, REFUND_SUCCEEDED, REFUND_FAILED, PAYMENT_FAILED, COMPLAINT_REPLY.

Bước 02. Khai báo mã template, locale, subjectTemplate, htmlBody, textBody, allowedVariables, trạng thái DRAFT/ACTIVE/ARCHIVED và version.

Bước 03. Backend giới hạn tập biến theo từng loại sự kiện, ví dụ {{customerName}}, {{bookingCode}}, {{propertyName}}, {{checkInDate}}, {{refundAmount}}, {{refundStatus}}.

Bước 04. ADMIN xem preview bằng dữ liệu giả lập được mask và kiểm tra biến thiếu; engine không thực thi script, expression tùy ý hoặc truy vấn DB từ template.

Bước 05. Kiểm tra HTML an toàn, escape biến theo ngữ cảnh HTML và tránh đưa URL ký MinIO vào template lưu trữ.

Bước 06. Publish chỉ khi biến bắt buộc đầy đủ, subject không rỗng, có fallback plain text và locale mặc định.

Bước 07. Ghi version immutable cho template được publish; ACTIVE chỉ trỏ tới version mới.

Bước 08. Sự kiện email được tạo với template_version hoặc snapshot nội dung để retry không thay nội dung bất ngờ.

Bước 09. ADMIN có thể gửi test đến danh sách email kiểm thử được cho phép, ghi audit và rate limit.

Bước 10. Thu hồi template không làm mất lịch sử email đã gửi; các sự kiện mới dùng version đang active.

Luồng thay thế, lỗi và ngoại lệ

– Template thiếu biến: reject publish hoặc email chuyển FAILED_VALIDATION, không gửi chuỗi {{...}} cho khách.

– Template tiếng Việt bị thiếu: fallback locale cấu hình, nhưng không tự thay thông tin pháp lý khác locale.

– Mẫu thay đổi khi email đang queue: retry theo version đã cố định.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Tuyệt đối không thay thế biến bằng raw string không escape trong HTML.

– Không đưa password, refresh token, token xác minh dạng đầy đủ vào log và audit.

– Mail tài khoản không gửi mật khẩu thuần; chỉ dùng link xác minh/đặt mật khẩu một lần có TTL.

Yêu cầu kỹ thuật và minh chứng

mail_templates/mail_template_versions/mail_events; renderer whitelist + context-aware escaping + fallback locale.

UC-32. Gửi email giao dịch tạo tài khoản, booking, thanh toán và hoàn tiền

Tác nhân: SYSTEM.

Tiền điều kiện: Có sự kiện nghiệp vụ đã commit hoặc phát ra outbox hợp lệ.

Luồng nghiệp vụ chính

Bước 01. Khi tạo tài khoản, commit user và verification_token an toàn; ghi outbox ACCOUNT_CREATED/EMAIL_VERIFICATION cùng transaction.

Bước 02. Khi booking CONFIRMED, ghi BOOKING_CONFIRMED event với bookingCode, ngày nhận/trả, tên cơ sở, thông tin liên hệ được phép.

Bước 03. Khi Fake Bank xác nhận thanh toán/thất bại, ghi PAYMENT_SUCCEEDED/PAYMENT_FAILED event theo trạng thái cuối hợp lệ.

Bước 04. Khi refund SUCCEEDED/FAILED, ghi REFUND_SUCCEEDED/REFUND_FAILED với mã refund, số tiền, lý do phù hợp.

Bước 05. Email worker claim outbox/mail_event bằng trạng thái QUEUED -> SENDING, dùng khóa dedupe theo eventType + businessId + recipient + templateVersion.

Bước 06. Nạp phiên bản template, bind variable whitelist từ snapshot theo nghiệp vụ, render subject/html/text và kiểm tra dữ liệu bắt buộc.

Bước 07. Gửi qua SMTP test hoặc nhà cung cấp mail; lưu provider_message_id, sent_at, attempt_count, error_code và trạng thái.

Bước 08. Nếu lỗi tạm thời, retry exponential backoff và jitter; nếu lỗi vĩnh viễn, FAILED/DLQ cho SUPPORT xử lý.

Bước 09. Người dùng xem trạng thái thông báo trong ứng dụng; không hiển thị bí mật trong mail hoặc log.

Bước 10. Khi nhấn gửi lại, tạo delivery attempt mới trong cùng logical email, không nhân đôi business event.

Luồng thay thế, lỗi và ngoại lệ

– DB đã commit nhưng SMTP timeout: không rollback user/booking/refund; giữ mail để retry/dedupe.

– Payment webhook trùng: chỉ một logical email PAYMENT_SUCCEEDED được tạo.

– Địa chỉ mail không hợp lệ: chuyển permanent failed và ghi lỗi có che PII.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Mail là side effect bất đồng bộ; không gọi SMTP trong transaction giữ khóa tồn kho.

– Mỗi mail có business correlation ID và snapshot data để điều tra.

– Test email tạo tài khoản, xác nhận booking, refund, callback duplicate và crash sau gửi trước khi cập nhật DB.

Yêu cầu kỹ thuật và minh chứng

Transactional outbox + mail queue + retry bounded; MailHog/Mailpit cho môi trường local.

UC-33. Nhắc email khi gần ngày check-in theo cấu hình

Tác nhân: SYSTEM/SCHEDULER.

Tiền điều kiện: Booking CONFIRMED, email đã xác minh, cấu hình reminder bật.

Luồng nghiệp vụ chính

Bước 01. ADMIN hoặc HOST được phân quyền chọn reminder.beforeCheckin với đơn vị MINUTES/HOURS/DAYS và cutoff, ví dụ 48 giờ trước giờ nhận phòng.

Bước 02. Ứng dụng chốt thời điểm dueAt = checkinScheduledAt − duration theo timezone property, lưu instant UTC để scheduler so sánh.

Bước 03. Job quét những booking CONFIRMED chưa check-in, chưa hủy, dueAt <= now, và chưa có email reminder logical cùng policy version.

Bước 04. Multiple scheduler instance dùng claim batch có khóa SKIP LOCKED hoặc lease DB, không phụ thuộc Redis lock làm bảo đảm duy nhất.

Bước 05. Job tạo email outbox CHECKIN_REMINDER với khóa unique bookingId + reminderType + reminderWindowCode + policyVersion.

Bước 06. Worker render mẫu với {{customerName}}, {{bookingCode}}, {{propertyName}}, {{checkInDate}}, {{checkInTime}}, {{propertyAddress}}, {{contactLink}}.

Bước 07. Gửi email; lưu sent_at và kết quả; hệ thống có thể gửi nhắc bổ sung nếu cấu hình có nhiều mốc rõ ràng.

Bước 08. Khi giờ check-in thỏa thuận thay đổi, tính lại dueAt của các reminder chưa gửi theo policy và vô hiệu nhắc không còn phù hợp.

Bước 09. Khi booking CANCELLED, EXPIRED hoặc CHECKED_IN, hủy tất cả reminder chưa gửi.

Bước 10. Job dashboard hiển thị số email sắp tới, số gửi thành công, backlog, độ trễ và lỗi gửi.

Luồng thay thế, lỗi và ngoại lệ

– Job trễ qua giờ check-in: cutoff policy quyết định bỏ qua, không gửi nhắc đã vô nghĩa.

– Mất kết nối SMTP: retry trong cửa sổ còn hữu ích, không spam sau hạn.

– Thời gian mùa hè (DST) và timezone property: test thay đổi offset; tránh tính ngày bằng phép cộng cố định 24 giờ trong local time.

– HOST đổi cấu hình khi mail đã SENT: không gửi lại cùng loại nếu không có mốc mới được định nghĩa.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Dedupe ở DB bằng unique key; TTL Redis không đủ bảo vệ gửi lặp.

– Không gửi email cho booking chưa xác nhận; không tự đổi booking status bởi job mail.

– Đo độ trễ scheduled_due_at tới sent_at.

Yêu cầu kỹ thuật và minh chứng

Scheduler + PostgreSQL claimed jobs/outbox, UTC storage/local timezone; Redis optional cache config, không lưu lịch độc quyền.

UC-34. Giám sát, xử lý lỗi và gửi lại email an toàn

Tác nhân: SUPPORT, ADMIN, SYSTEM.

Tiền điều kiện: Có mail_event và các delivery_attempt.

Luồng nghiệp vụ chính

Bước 01. SUPPORT tra cứu trạng thái email theo event, bookingCode, khoảng ngày, template, loại lỗi, số lần retry và providerMessageId.

Bước 02. Hệ thống che địa chỉ email nếu vai trò không được xem đầy đủ; audit truy cập nội dung.

Bước 03. Mail worker phân loại SMTP 4xx tạm thời so với lỗi địa chỉ/validation vĩnh viễn; retry theo policy.

Bước 04. Khi quá retry hoặc TTL gửi đã hết, chuyển DLQ, ghi reason và cảnh báo dashboard.

Bước 05. SUPPORT chọn resend, backend xác minh email vẫn có ý nghĩa nghiệp vụ, booking chưa hủy và user vẫn đủ điều kiện.

Bước 06. Gửi lại cùng logical notification với delivery_attempt mới và một idempotencyKey operator action.

Bước 07. Có thống kê queued/sent/failed/retried theo ngày và loại mail; xem tỉ lệ lỗi không lộ thông tin nội dung.

Bước 08. Khi nhà cung cấp nhận request nhưng response timeout, trạng thái UNKNOWN; tra providerMessageId nếu hỗ trợ, không kết luận chưa gửi.

Luồng thay thế, lỗi và ngoại lệ

– Không cho gửi lại EMAIL_VERIFICATION link đã hết hạn mà giữ nguyên token cũ; phải tạo token mới theo quy tắc an toàn.

– Mail refund báo thành công chỉ được gửi sau khi refund ở trạng thái SUCCEEDED thật.

– Hai support cùng bấm resend: concurrency/idempotency tránh trùng hành động.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Chấp nhận ngữ nghĩa at-least-once của email; dedupe ở mức logical event, delivery có thể trùng nếu provider không hỗ trợ idempotency.

– Không đưa API key SMTP vào DB cấu hình hiển thị hoặc báo cáo.

Yêu cầu kỹ thuật và minh chứng

mail_delivery_attempts, mail_dead_letters; queue claiming/lease; retry/backoff/audit.

UC-35. Fake Bank độc lập: số dư, thanh toán, webhook và đối soát

Tác nhân: CUSTOMER, Fake Bank, SYSTEM, ADMIN (chỉ xem).

Tiền điều kiện: Có booking HELD/PENDING_PAYMENT chưa hết hạn và paymentIntent hợp lệ.

Luồng nghiệp vụ chính

Bước 01. Booking gửi sang Fake Bank qua HTTP nội bộ với paymentIntentId, bookingCode, amount, currency, merchantId, idempotencyKey và signature HMAC giả lập.

Bước 02. Fake Bank xác thực chữ ký, kiểm tra merchant, merchantReference, amount và unique merchantId + idempotencyKey.

Bước 03. Fake Bank tạo bank_transaction trạng thái INITIATED; cùng key/cùng requestHash trả transaction cũ, cùng key/khác payload trả IDEMPOTENCY_CONFLICT.

Bước 04. Kiểm tra tài khoản ngân hàng giả lập, số dư đủ và giới hạn transaction; chuyển PROCESSING.

Bước 05. Trong một transaction riêng của Fake Bank, lock account/conditional debit, ghi double-entry ledger debit user/credit merchant và bank_transaction SUCCEEDED.

Bước 06. Fake Bank phản hồi đồng bộ là ACCEPTED/PROCESSING hoặc kết quả cuối, đồng thời phát webhook có providerTxnId, amount, currency, status, eventId và HMAC.

Bước 07. Payment service nhận webhook, xác minh signature, timestamp/replay window, eventId unique, kiểm tra booking/hold và số tiền khớp.

Bước 08. Trong transaction PostgreSQL của hệ thống đặt phòng, lock booking và payment, cập nhật SUCCEEDED, booking CONFIRMED, giữ nguyên inventory và ghi outbox email.

Bước 09. Nếu webhook lặp, chỉ ack, không cộng số dư hoặc đổi booking thêm; nếu callback muộn sau EXPIRED chuyển NEEDS_REFUND/REVIEW.

Bước 10. Job đối soát định kỳ so sánh transaction Fake Bank với payment local theo providerTxnId; tạo repair task/refund khi sai lệch.

Bước 11. Admin có thể dùng kịch bản cấu hình: success, insufficient balance, DECLINED, processing timeout, callback lost, duplicate, delayed, DB outage.

Luồng thay thế, lỗi và ngoại lệ

– Fake Bank debit thành công nhưng HTTP timeout: Payment local không được đánh dấu FAILED ngay; trạng thái UNKNOWN/RECONCILING, truy vấn lại bằng idempotencyKey.

– Fake Bank debit thất bại trong DB của Fake Bank: rollback ledger/account và trả FAILED, booking giữ chỗ hoặc hủy theo chính sách hết hạn.

– Booking DB lỗi ngay sau bank captured: không gọi rollback DB để “hoàn tiền”; tạo compensation refund sau đối soát.

– Tài khoản giả lập không đủ tiền: không thay đổi ledger và không gửi webhook SUCCEEDED.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Bất biến: tổng debit/credit ledger cân bằng, một idempotencyKey không bị trừ hai lần; cùng providerTxnId chỉ một payment captured.

– Fake Bank phải chạy tiến trình và DB schema riêng hoặc ít nhất transaction boundary riêng để mô phỏng phân tán.

– Webhook phải không tin status/amount từ client; chỉ nhận chữ ký từ Fake Bank.

Yêu cầu kỹ thuật và minh chứng

FakeBank account + ledger entries + transactions + webhook_deliveries; transactional outbox và reconciliation; không sử dụng tiền thật.

UC-36. Rollback cục bộ và bù trừ khi thanh toán lỗi

Tác nhân: SYSTEM, CUSTOMER, SUPPORT.

Tiền điều kiện: Có payment intent và booking giữ phòng; quy tắc timeout/hủy rõ ràng.

Luồng nghiệp vụ chính

Bước 01. Khởi tạo booking/hold trong transaction local; rollback tất cả booking_nights/hold nếu insert hoặc kiểm tra tồn kho lỗi trước commit.

Bước 02. Sau khi local booking commit, gửi paymentIntent đến Fake Bank ngoài transaction DB đang giữ lock.

Bước 03. Nếu Fake Bank xác nhận FAILED mà chưa debit: transaction local cập nhật payment FAILED; chính sách quyết định cho retry trong thời hạn giữ chỗ hay hủy/giải phóng hold.

Bước 04. Nếu payment timeout/UNKNOWN: không giải phóng hold như thể chắc chắn thất bại; chuyển RECONCILING và hỏi trạng thái bank theo idempotency key, giữ hold đến cutoff rõ ràng.

Bước 05. Nếu Fake Bank SUCCEEDED và cập nhật booking trong local transaction thất bại: rollback riêng cập nhật local, ghi sự kiện lỗi bền vững và reconciliation tìm bank transaction đã captured.

Bước 06. Đối soát thử cập nhật booking nếu hold còn hiệu lực, tồn kho an toàn và trạng thái phù hợp; nếu không, tạo compensation REFUND_REQUESTED.

Bước 07. Fake Bank xử lý refund idempotent theo refundKey, lock ledger, ghi bút toán ngược theo số tiền cho phép; phản hồi webhook refund.

Bước 08. Payment service cập nhật refund SUCCEEDED; booking CANCELLED/EXPIRED phù hợp, inventory được giải phóng đúng một lần và outbox tạo mail hoàn tiền.

Bước 09. Với các bước chưa chạy (mail, notification, check-in), chỉ thực hiện sau khi trạng thái cuối commit; không “undo” mail đã gửi mà gửi mail hiệu chỉnh nếu cần.

Bước 10. Nếu refund thất bại/timeout, giữ REFUND_PENDING/NEEDS_ATTENTION, retry có giới hạn, không coi workflow hoàn tất.

Bước 11. Mọi bước ghi operationId, idempotencyKey, eventId, before/after status, error code, attempt count để audit và phát hiện inconsistency.

Luồng thay thế, lỗi và ngoại lệ

– Bank captured nhưng local DB mất kết nối dài: ghi nhận trạng thái phía bank, tiếp tục reconcile khi DB phục hồi; không tự coi tiền chưa bị trừ.

– Callback success cạnh tranh với hold-expiry job: cùng lock order booking -> payment -> inventory; một transition thắng, nhánh còn lại bù trừ.

– User bấm pay hai lần: cùng paymentIntentId trả một kết quả; cùng booking nhưng intent mới chỉ khi intent trước kết thúc/hết hiệu lực theo quy tắc.

– Partial refund: tổng refund SUCCEEDED không vượt capturedAmount; duplicate refund request không hoàn hai lần.

Quy tắc nghiệp vụ và yêu cầu dữ liệu

– Từ "rollback" chỉ áp dụng transaction của một DB; hai DB khác nhau xử lý bằng saga/compensation/reconciliation.

– Định nghĩa rõ thời điểm unlock inventory và khi nào gửi mail xác nhận/hoàn tiền.

– Test fault injection ở mọi điểm: trước debit, sau debit, trước commit booking, sau commit booking, mất webhook, timeout refund.

Yêu cầu kỹ thuật và minh chứng

Saga orchestration theo state machine + local ACID transaction + DB constraints + outbox + reconciliation.

4. Vòng đời trạng thái và quan hệ nghiệp vụ

Thực thể	Vòng đời chính	Điều kiện/nhánh ngoại lệ
Property	DRAFT → PUBLISHED → PAUSED → ARCHIVED	Publish cần đủ phòng/giá/ảnh/chính sách; PAUSED dừng bán mới.
Booking	PENDING_PAYMENT → CONFIRMED → CHECKED_IN → COMPLETED	PENDING_PAYMENT → EXPIRED/CANCELLED; CONFIRMED → CANCELLATION_PENDING/REFUND_PENDING → CANCELLED; cần REVIEW khi sai lệch.
Payment	INITIATED → PROCESSING → SUCCEEDED/FAILED/TIMED_OUT	Success không được quay về failed; webhook trùng không đổi hiệu ứng.
Refund	REQUESTED → PROCESSING → SUCCEEDED/FAILED	Refund không vượt số tiền đã thực thu; retry giữ cùng khóa nghiệp vụ.
Room	ACTIVE / INACTIVE / MAINTENANCE	Ngừng bán không xóa hoặc vô hiệu booking tương lai.
Media	PENDING → READY → DELETING/DELETED	PENDING_EXPIRED và FAILED được dọn qua job.

4.1. Thứ tự khóa và nhất quán

- Luồng booking, callback thanh toán, hủy và job hết hạn phải thống nhất thứ tự khóa bản ghi (ví dụ booking trước allocation, sau đó payment nếu cùng thao tác) và được kiểm chứng thực tế tránh deadlock.
- DB constraint bảo đảm không chồng lấn; state machine bảo đảm transition hợp lệ; idempotency bảo đảm retry an toàn. Ba lớp giải quyết ba dạng lỗi khác nhau.

- Định nghĩa rõ trạng thái nào chiếm kho: PENDING_PAYMENT còn hiệu lực, CONFIRMED, CHECKED_IN; EXPIRED/CANCELLED không chiếm kho. Nếu muốn hủy đang xử lý hoàn tiền vẫn giữ phòng, cần quy định cụ thể theo sản phẩm.
- Mỗi thay đổi trạng thái lưu booking_status_history gồm fromStatus, toStatus, actor, reason, at và correlationId.

5. Mô hình dữ liệu khuyến nghị và ràng buộc

Nhóm bảng	Trường và constraint quan trọng
identity	users(email unique, status, password_hash, token_version); roles, permissions, user_roles, role_permissions; refresh_tokens(hash, expires_at, revoked_at)
property	properties(owner_id, timezone, status); room_types(property_id, capacity); rooms(room_type_id, room_code, status), unique(property_id, room_code) theo thiết kế
pricing/media	room_rates(room_id hoặc room_type_id, stay_date, amount, currency) unique theo date; amenities; media_assets(object_key unique, status, checksum, size)
booking	bookings(customer_id, room_id, check_in, check_out, status, expires_at, total_amount, currency, version); booking_nights unique(booking_id, stay_date); snapshot_cancel_policy
inventory	room_allocations(room_id, booking_id, date range/status) và exclusion constraint phù hợp, hoặc bảng tồn theo ngày + conditional update; chọn một mô hình nhất quán
payment	payments(booking_id, provider_reference unique, idempotency_key, amount, status); refunds(payment_id, provider_ref, idempotency_key, amount, status)
support	reviews(booking_id unique, rating, status); notifications(event_id, recipient_id unique pair); booking_status_history; audit_logs; outbox_events; idempotency_records(scope,key unique, request_hash)

5.1. Lựa chọn tồn kho

Ưu tiên mô hình phòng vật lý (room_id) để đặc tả gần với hệ thống khách sạn nhỏ và kiểm chứng chồng lấn chính xác. PostgreSQL có thể dùng exclusion constraint trên daterange cho các allocation còn chiếm dụng; khi constraint phụ thuộc trạng thái động/hạn giữ, cần thiết kế lại để tránh điều kiện constraint không ổn định theo thời gian. Phương án thay thế là khóa room row bằng PESSIMISTIC_WRITE rồi kiểm tra overlap và insert trong cùng transaction. Không trộn hai cơ chế ở các use case khác nhau mà thiếu quy tắc thống nhất.

Nếu chuyển sang mô hình bán theo số lượng RoomType, áp dụng inventory_daily(room_type_id, stay_date, available_count), cập nhật có điều kiện available_count > 0 cho tất cả đêm, kiểm tra số hàng cập nhật và rollback toàn bộ nếu thiếu bất kỳ đêm nào. Luồng trả lại tồn cũng cần idempotency.

6. Thiết kế kỹ thuật, transaction và tích hợp

6.1. Công nghệ và trách nhiệm

Thành phần	Sử dụng trong hệ thống	Lưu ý triển khai
Java 21 + Spring Boot 3.x	API nghiệp vụ, validation, orchestration	Application service làm ranh giới use case; tránh controller chứa business logic.
Spring Security + JWT	Authentication, RBAC, data scope	Kiểm tra quyền đối tượng tại service/query, không chỉ endpoint.
Spring Data JPA + EntityManager	CRUD, query động, locks, batch	DTO projection, @Version, EXPLAIN ANALYZE, tránh N+1.
PostgreSQL + Flyway	Nguồn sự thật; schema migration	Constraint + index + transaction là tầng bảo đảm cuối.
Redis	Cache search/detail và rate limiting	TTL/invalidation, giảm tải; không bảo đảm tồn kho.
MinIO	Ảnh property/room	Presigned URL và cleanup object mồ côi.
Quartz/Spring Scheduler	Job hết hạn, reconciliation, cleanup	Batch, claim/locking, chạy lại an toàn.
Actuator/Micrometer/Prometheus/Grafana	Metrics, health và dashboard	Tránh tag userId/bookingId làm metric cardinality cao.
Docker Compose + Testcontainers	Môi trường tích hợp và integration tests	Tách .env.example, secrets ngoài source code.

6.2. Tình huống transaction và lựa chọn xử lý

Kịch bản	Cách bảo đảm	Cách chứng minh
Ghi booking xong, ghi đêm thất bại	@Transactional rollback toàn bộ booking + hold	Inject lỗi DB, kiểm tra không còn allocation
Hai khách đặt cùng phòng	Row lock hoặc exclusion constraint/conditional update	Bắn 50–100 request đồng thời, chỉ một hợp lệ
Retry request booking	Unique idempotency scope + key + requestHash	Cùng key cùng payload không sinh bản ghi mới

Webhook và job hết hạn đua nhau	State check và lock cùng quy ước	Đồng bộ luồng chạy đồng thời, kiểm tra một trạng thái thắng
Payment thành công, DB timeout	Webhook retry, provider reference, reconciliation	Inject DB timeout và chạy đối soát
MinIO upload xong, DB rollback	PENDING + finalize + orphan cleanup	Giả lập DB down sau upload
Self-invocation @Transactional	Tách bean/call qua proxy phù hợp	Integration test xem commit/rollback thực tế
Audit REQUIRES_NEW	Phân biệt attempt vs success, tránh audit sai	Rollback tx ngoài, kiểm tra ý nghĩa audit
N+1, pagination chậm	Projection/EntityGraph, query plan + index	Đếm SQL và EXPLAIN ANALYZE trước/sau

6.3. Nguyên tắc áp dụng Spring AOP

- Audit các thao tác nhạy cảm có thể dùng @Audited kết hợp aspect; không đưa thuật toán chuyển trạng thái cốt lõi vào aspect.
- Đo độ trễ use case bằng Micrometer Timer/annotation phù hợp, kết hợp correlationId từ filter/interceptor.
- @Transactional qua Spring proxy có bẫy self-invocation; propagation REQUIRES_NEW phải nêu rõ lý do và ảnh hưởng connection pool.
- Không retry mù thao tác không idempotent; retry network I/O cần timeout, backoff và chính sách lỗi.

6.4. Fake Bank - cấu trúc giải pháp và ranh giới ACID

- Fake Bank là ứng dụng hoặc module triển khai độc lập qua HTTP với database/schema tách transaction: bank_accounts(id, balance, currency, version), bank_transactions(id, merchant_id, merchant_reference, idempotency_key, request_hash, amount, status, provider_txn_id), bank_ledger_entries(txn_id, account_id, debit, credit), bank_webhook_outbox(event_id, txn_id, event_type, attempts).
- Không xử lý thông tin ngân hàng thật; tạo các tài khoản test có số dư giả lập. Khi debit, khóa dòng tài khoản và kiểm tra đủ balance; commit ledger + transaction cùng một local DB transaction phía bank; invariant tổng debit bằng tổng credit.
- Phía Booking có payment_intents, payments, payment_events, refunds, reconciliation_cases, outbox_events. Unique (merchant_id, idempotency_key), (provider_txn_id), (provider_event_id), (refund_key) là các chốt bắt buộc.
- Phân biệt thanh toán FAILED chắc chắn chưa thu, UNKNOWN chưa xác định, SUCCEEDED đã ghi sổ bank. Không chuyển UNKNOWN sang FAILED do client timeout đơn thuần.
- Timeout Fake Bank -> tra cứu GET /fake-bank/transactions/by-idempotency-key; bank success/booking fail -> reconcile hoặc refund; mọi nhánh chỉ cập nhật trạng thái dựa trên transition hợp lệ.
- Email và notification chỉ được sinh từ outbox sau local state commit. Nếu refund success nhưng mail lỗi, giữ refund success và retry mail; không hoàn tiền lần thứ hai.

Điểm lỗi giả lập	Transaction cần rollback / trạng thái	Tiếp tục xử lý
DB booking lỗi trước commit hold	Rollback booking + booking_nights + inventory allocation	Trả lỗi, không tạo payment intent
Fake Bank không đủ tiền / DECLINED	Rollback debit + ledger phía Fake Bank	Booking vẫn HELD hoặc hủy theo chính sách; cho thanh toán lại nếu còn hạn
HTTP timeout sau Fake Bank debit	Không rollback bank vì đã commit; local payment UNKNOWN	Query bank bằng idempotencyKey, reconcile
Bank success, Booking DB update lỗi	Rollback transaction cập nhật booking ở DB Booking	Recovery bằng webhook replay/reconcile; nếu không thể confirm thì refund
Webhook success đến khi booking EXPIRED	Không confirm booking quá hạn	Tạo NEEDS_REFUND và bút toán đảo
Refund bank đã thành công, cập nhật local lỗi	Không refund lại bằng key mới	Query/replay idempotent, update local rồi gửi email
Email gửi lỗi sau refund SUCCEEDED	Không rollback refund / booking	Retry mail outbox, theo dõi DLQ
POST /api/v1/payments/intents | GET /api/v1/payments/{id}	Tạo/tra trạng thái payment intent idempotent	CUSTOMER owner
POST /fake-bank/api/v1/transactions/debit	Fake Bank trừ tiền giả lập theo idempotency	Service authentication
GET /fake-bank/api/v1/transactions/by-key	Truy vấn giao dịch bank để đối soát	Service authentication
POST /fake-bank/api/v1/refunds	Bù trừ refund idempotent	Service authentication
POST /api/v1/payments/fake-bank/webhook	Callback signed, replay protected	Signed Fake Bank
GET /api/v1/analytics/overview	KPI dashboard theo scope	HOST/ADMIN
POST /api/v1/analytics/query	Báo cáo đa filter, groupBy whitelist	HOST/ADMIN
POST /api/v1/analytics/exports	Tạo job xuất báo cáo lớn	HOST/ADMIN
GET /api/v1/analytics/exports/{id}	Trạng thái/kết quả export	Owner scope
POST /api/v1/conversations	Mở hội thoại theo property/booking	CUSTOMER
GET /api/v1/conversations | GET /api/v1/conversations/{id}/messages	Inbox/cursor message	Participant/SUPPORT scope
POST /api/v1/conversations/{id}/messages	Gửi chat idempotent	Participant
POST /api/v1/conversations/{id}/complaints	Tạo phản ánh/ticket	CUSTOMER participant
POST /api/v1/bookings/{id}/checkin-requests	Đề nghị giờ check-in	Booking owner
PATCH /api/v1/checkin-requests/{id}/respond	HOST phản hồi/đề nghị giờ khác	HOST scope

GET/PATCH /api/v1/admin/settings	Xem/sửa cấu hình mật khẩu & nhắc mail	ADMIN
GET/POST /api/v1/admin/email-templates	Danh sách/tạo mẫu email	ADMIN
POST /api/v1/admin/email-templates/{id}/publish	Xuất bản phiên bản template	ADMIN
GET /api/v1/admin/mail-events | POST /api/v1/admin/mail-events/{id}/retry	Tra cứu lỗi & retry mail	SUPPORT/ADMIN

6.5. Kiến trúc dữ liệu thống kê và tối ưu hóa PostgreSQL + Redis

- Dimension được phép: propertyId, roomTypeId, bookingStatus, paymentStatus, refundStatus, guestSegment, sourceChannel, stayDate, createdDate, capturedDate, dayOfWeek và priceBand; whitelist metric và dimensions thay cho SQL string từ đầu vào.
- Chọn read model theo mức tải: truy vấn chi tiết trực tiếp PostgreSQL cho dữ liệu nhỏ; CTE/filtered aggregate cho đa điều kiện; bảng fact theo ngày để tránh quét bảng bookings/payments lớn lặp lại; các thay đổi được ghi theo event id để incremental update idempotent.
- Định nghĩa rõ facts: booking_fact 1 dòng mỗi booking; booking_night_fact 1 dòng mỗi đêm; payment_fact 1 dòng mỗi capture; refund_fact 1 dòng mỗi refund thực thi; occupancy_daily_fact 1 dòng theo phòng/ngày. Không join facts khác grain rồi tính SUM trực tiếp.
- Index dựa workload: (property_id, created_at, status), (room_id, stay_date), (status, captured_at), (payment_id, status) với partial index chỉ khi filter lặp lại. Dùng EXPLAIN (ANALYZE, BUFFERS) đo thực tế và so sánh trước/sau; không coi tạo nhiều index mặc nhiên tốt.
- Các báo cáo có điều kiện phức tạp phải chứng minh dữ liệu đúng khi filter kết hợp 5–8 điều kiện, groupBy 2–3 chiều, pagination, khoảng ngày xa và tổ hợp trạng thái. Đặt statement_timeout, giới hạn range, rows và số truy vấn concurrent.
- Redis key mẫu analytics:v3:{scopeHash}:{metricHash}:{filterHash}:{granularity}:{dataVersion}; không đưa PII thô vào key. TTL gợi ý 60–300 giây tùy KPI, giá trị thực tế được cấu hình.
- Invalidation bằng event sau commit hoặc version counter theo property/tenant; có protection cache stampede (single-flight/short lease) và stale policy công khai. Mỗi response có asOf, granularity, dataFreshness, fromCache.
- Báo cáo dữ liệu lịch sử có thể cache lâu hơn; chỉ số giao dịch mới phát sinh cần TTL ngắn hoặc event invalidation. Admin thay đổi quyền phạm vi dữ liệu phải thay fingerprint cache.

6.6. Hội thoại, phản ánh và lịch check-in

- Bảng conversations(id, property_id, booking_id nullable, customer_id, host_id, type, status, last_message_seq), conversation_participants, messages(conversation_id, seq, sender_id, client_message_id, type, body, asset_id, created_at), read_receipts, complaint_tickets, checkin_requests.

- Không cho truy vấn messages theo conversation_id đơn thuần: bắt buộc xác minh principal là participant hiện tại hoặc SUPPORT được phân công; masking dữ liệu và audit khi tham gia xử lý phản ánh.
- Mỗi lần gửi, tạo message và outbox notification cùng DB transaction; giao realtime chỉ là tăng trải nghiệm, client phục hồi bằng GET messages?cursor=... và không mất thông điệp.
- Phản ánh có loại, mức độ, lịch sử chuyển trạng thái OPEN → ACKNOWLEDGED → IN_PROGRESS → RESOLVED/CLOSED; nếu không đủ thông tin, yêu cầu bổ sung thay vì tự đóng.
- Check-in request là lịch hẹn tách khỏi booking.status; lịch gửi email reminder tham chiếu agreed_checkin_at nếu đã có, nhưng nghiệp vụ check-in thực tế phải được xác nhận riêng.

6.7. Email template, policy cấu hình và scheduler gửi nhắc

- Bảng config_definitions, config_values(scope, key, value, value_type, effective_at, version), config_change_logs. Khóa chính sách: auth.password.minLength, auth.password.maxLength, checkin.reminder.enabled, checkin.reminder.beforeMinutes, mail.retry.maxAttempts, mail.retry.backoffSeconds, mail.defaultLocale.
- Bảng email_templates(template_code, locale, active_version), email_template_versions(subject, html_body, text_body, allowed_variables, published_at), mail_events(event_id, business_id, type, template_version, recipient, payload_snapshot, status), mail_delivery_attempts, scheduled_reminders.
- Email nhắc có cấu hình cụ thể trước bao nhiêu phút/giờ/ngày. Mỗi booking/reminderType/policyVersion có unique; chỉ gửi CONFIRMED trước cutoff và chưa CHECKED_IN/CANCELLED. Job claim DB theo batch để nhiều instance không gửi trùng.
- Ví dụ ACCOUNT_CREATED: Xin chào {{customerName}}, tài khoản {{maskedEmail}} đã được tạo; bấm {{verificationLink}} để xác minh. Không nhúng password. Ví dụ REFUND_SUCCEEDED: Booking {{bookingCode}} đã hoàn {{refundAmount}} {{currency}}, mã hoàn {{refundCode}}.
- Template biến phải thuộc danh sách cho phép theo event, escape HTML, kiểm tra missing variables. Khi template publish mới, mail đã xếp hàng giữ nguyên template_version và dữ liệu snapshot.
- Thời điểm áp dụng cấu hình: thay đổi minLength tác động vào đăng ký/đổi mật khẩu từ effectiveAt; thay đổi reminder áp dụng cho reminder chưa gửi theo rule EFFECTIVE_FOR_UNSENT, không gửi lặp reminder đã hoàn tất.
- Vận hành: SMTP fake (Mailpit/MailHog), timeouts, retry + DLQ + dedupe, kiểm tra asOf và metric queue_delay, send_success_rate, checkin_reminder_late_total.

7. API contract gợi ý theo phân hệ

API	Chức năng	Quyền
POST /api/v1/auth/register | /verify	Đăng ký / xác minh email	Public
POST /api/v1/auth/login | /refresh | /logout	Token lifecycle	Public / Authenticated
GET/PATCH /api/v1/users/me	Xem/cập nhật hồ sơ	Authenticated

POST/PATCH /api/v1/host/properties	Tạo/cập nhật property	HOST + owner scope
POST/PATCH /api/v1/host/room-types | /rooms | /rates	Quản lý catalog và giá	HOST + owner scope
POST /api/v1/media/uploads | /finalize	Upload ảnh MinIO	HOST + owner scope
GET /api/v1/properties | /rooms/search	Search property, phòng	Public
GET /api/v1/rooms/{id}/quote	Chi tiết & quote theo ngày	Public
POST /api/v1/bookings	Tạo booking/hold	CUSTOMER
GET /api/v1/bookings/me | /{id}	Danh sách/chi tiết theo quyền	CUSTOMER/HOST/SUPPORT
POST /api/v1/bookings/{id}/cancel	Xác nhận hủy	Owner/SUPPORT theo quyền
POST /api/v1/payments | /webhook	Thanh toán mô phỏng/callback	CUSTOMER / Verified webhook
POST /api/v1/refunds/{id}/reconcile	Đối soát thủ công có kiểm soát	SUPPORT/ADMIN
POST /api/v1/host/bookings/{id}/check-in | /check-out	Nhận/trả phòng	HOST + owner scope
POST /api/v1/bookings/{id}/reviews	Viết đánh giá	Booking owner
GET/PATCH /api/v1/notifications	Danh sách/đã đọc	Authenticated
GET /api/v1/host/reports	Báo cáo theo property	HOST + owner scope
GET /api/v1/admin/audit | /users | /roles	Audit/quản trị	ADMIN / permission

Lỗi JSON thống nhất: timestamp, traceId, code, message, details; dùng HTTP 400 cho input sai, 401/403 cho auth, 404 với tài nguyên ẩn, 409 cho trạng thái hoặc xung đột, 429 cho rate limit. Các mã: VALIDATION_ERROR, ROOM_UNAVAILABLE, PRICE_CHANGED, BOOKING_EXPIRED, CANCELLATION_NOT_ALLOWED, FORBIDDEN_SCOPE, IDEMPOTENCY_CONFLICT, PAYMENT_REQUIRES_REVIEW. Không trả stack trace cho client.

8. Chất lượng, kiểm thử và quan sát vận hành

8.1. Bộ tình huống kiểm thử xuyên phân hệ

- Tài khoản: đăng ký cùng email đồng thời; đăng nhập sai/khóa; refresh rotation; user cố tự gán role; token hết hạn; kiểm tra 401/403.
- Property/media: HOST A thay ID sang property HOST B; publish thiếu ảnh/giá; đóng phòng đang có booking; upload file không hợp lệ; object mồ côi sau lỗi DB.
- Search/pricing: checkOut bằng checkIn; DST/timezone theo địa phương; 3 đêm mỗi đêm giá khác nhau; thiếu giá một đêm; sort injection; phân trang ổn định.

- Booking: 100 request cùng phòng và đêm; retry cùng idempotencyKey; cùng key payload khác; rollback một booking_night; lock timeout/deadlock.
- Payment/expiry: webhook thành công trùng 5 lần; callback muộn sau EXPIRED; callback đúng lúc job claim; success provider + lỗi DB; job chạy hai instance.
- Cancellation/refund: hủy trước/sau deadline; hủy trước thanh toán; đã refund một phần; callback refund trùng; refund vượt capturedAmount.
- Check-in/review: check-in booking sai ngày/sai cơ sở; check-out chưa check-in; đánh giá khi chưa hoàn tất; hai request cùng tạo review.
- Observability: dashboard có latency p50/p95/p99, lỗi theo route, HikariCP pool, booking_conflict_total, hold_expired_total, scheduler failures, Redis hit/miss.

8.2. Phi chức năng và phương pháp đo

- Tính đúng đắn: không bán trùng phòng trong mọi concurrency test; không sinh payment/refund trùng khi retry; không lộ dữ liệu khác quyền.
- Hiệu năng: đo p95/p99 search và booking theo dataset, cấu hình phần cứng và concurrent users đã ghi nhận; không tuyên bố SLA nếu chưa có số liệu thử tải phù hợp.
- Khả năng phục hồi: phân biệt lỗi tạm thời và lỗi không thể retry, có reconciliation/job idempotent và dashboard cho backlog.
- An toàn: giới hạn rate; sanitize log, bảo vệ actuator; quản lý secrets, backups và migration rollback plan.
- Chất lượng truy vấn: ghi kế hoạch EXPLAIN ANALYZE cho search/aggregation; index dựa trên workload thực tế, hạn chế N+1.

8.3. Kịch bản kiểm thử bổ sung cho từng luồng mới

- SQL Analytics: dashboard phân biệt số đêm/số booking; JOIN booking_nights không nhân bản tiền; so sánh kỳ và timezone; filter 6–8 điều kiện, bảng lớn, explain plan trước/sau index và pre-aggregation.
- Redis Analytics: cache HIT/MISS, hết TTL, dữ liệu thay đổi trước TTL, cache stampede 100 request, Redis down fallback; user HOST A không nhận cache HOST B dù filter giống nhau.
- Fake Bank: trừ tiền hai lần cùng idempotency key, một key khác payload, thiếu tiền, HMAC sai, timeout sau debit, webhook lặp 10 lần, callback late, bank success + DB booking fail + refund.
- Tin nhắn: hai clientMessageId trùng, gửi cùng lúc, sai participant, HOST đổi owner property, đánh dấu đã đọc race, recovery khi socket rớt, upload ảnh không hợp lệ.
- Phản ánh và check-in: HOST trả lời ticket sai cơ sở; khách yêu cầu giờ khác; HOST phản hồi đồng thời; booking hủy ngay trước giờ nhắc; check-in thực tế khác giờ hẹn.
- Policy: đổi password.minLength và maxLength; xác minh policy chỉ áp dụng từ effectiveAt, không leak mật khẩu; config multi-instance bị cache stale.
- Template: biến hợp lệ, biến thiếu, escape HTML, locale fallback, publish version mới khi mail cũ đang queue, HTML injection trong customerName.
- Mail: xác minh tài khoản, booking confirmed, refund succeeded, nhắc 48 giờ; job nhiều instance; SMTP timeout sau gửi; retry quá hạn; policy reminder thay đổi, không gửi mail trùng.

9. Kinh nghiệm kỹ thuật đúc kết sau khi triển khai

01. Thiết kế domain trước controller: Các quy tắc giá, giữ phòng, điều kiện hủy và trạng thái phải được viết thành bất biến/test. Nếu chỉ CRUD thì không bảo đảm tính đúng khi có cạnh tranh.

02. Transaction không phải khóa toàn bộ hệ thống: Chọn điểm bắt đầu/commit ngắn; chỉ bao dữ liệu PostgreSQL. Redis, MinIO, Fake Bank không rollback cùng DB nên cần trạng thái trung gian và compensation.

03. Concurrency cần kiểm chứng: SELECT rồi INSERT không đủ khi hai transaction chạy song song; phải mô phỏng request cạnh tranh và đo lock wait/deadlock.

04. Idempotency là hợp đồng API: Cùng khóa/cùng payload cho cùng kết quả; cùng khóa/khác payload bị từ chối. Unique DB constraint là chốt an toàn.

05. Phân biệt trạng thái ở các aggregate: Booking CONFIRMED và Payment SUCCEEDED liên quan nhưng không phải một cột status dùng chung; callback tới muộn là vấn đề nghiệp vụ thực.

06. Không tin nguồn dữ liệu phía client: Server tự tính tiền, owner scope, tồn kho và quyền truy cập. Số tiền trong request chỉ có vai trò đối chiếu.

07. Cache giúp nhanh nhưng tạo nợ nhất quán: Cache invalidation, TTL, version và stale policy cần công bố rõ; search cache không có quyền đặt phòng.

08. Tài liệu dữ liệu giúp tránh lỗi lịch sử: Snapshot từng đêm/chính sách, numeric currency, UTC timestamp và local stay_date giúp truy vết sai lệch.

09. JPA cần hiểu SQL thật: N+1, dirty checking, flush/clear, isolation, locks và projection ảnh hưởng trực tiếp đến độ trễ/đúng đắn.

10. Security nhiều lớp: SecurityFilterChain bảo vệ endpoint, method security hạn chế chức năng, query/service kiểm tra chủ sở hữu tránh IDOR.

11. Side effect phải hồi phục được: Outbox, reconciliation, cleanup object mồ côi và batch claim giúp chạy lại sau crash mà không nhân đôi hiệu ứng.

12. Quan sát trước khi tối ưu: Trace/correlationId + metric + DB query plan cho biết hệ thống chậm ở application, pool, DB lock hay cache.

13. Contract và test chung quan trọng hơn chia đều số endpoint: Phân công theo ownership aggregate, thống nhất state/API/event contract và dùng integration test xuyên phần việc để tránh lỗi ghép hệ thống.

9.1. Kinh nghiệm bổ sung từ thống kê, Fake Bank, chat và email

- Báo cáo KPI không thể đúng nếu thiếu định nghĩa grain và thời điểm: COUNT booking, SUM room nights và dòng tiền theo captured_at là ba thước đo khác nhau.
- Tối ưu SQL phải dựa trên query plan và tải mẫu. Một truy vấn động 8 filter với index sai có thể chậm hơn cách tổng hợp fact theo ngày; Redis không sửa được SQL tính sai.
- Cache analytics đa tenant cần scope-aware key/version; TTL chỉ kiểm soát thời gian lỗi thời chứ không đảm bảo dữ liệu luôn mới. Cần công bố asOf.

- Fake Bank giả lập cần ghi ledger thật trong DB mô phỏng để học điều kiện bất biến tiền; trả thành công giả mà không lưu sổ không tạo được tình huống bù trừ thực tế.
- Transaction rollback dừng ở một DB. Khi ngân hàng đã thu, Booking phải quyết định xác nhận hoặc refund bù trừ. Trạng thái UNKNOWN cần reconciliation chứ không retry debit với key mới.
- Callback, mail và message đều có thể nhận/gửi lặp do timeout. Dùng khóa idempotent ở dữ liệu bền vững và retry theo state machine, không coi một lần gọi HTTP là duy nhất.
- Email template cần immutable published version và snapshot variable, nếu không một mail retry có thể thay đổi nội dung pháp lý khi admin vừa sửa template.
- Scheduler nhắc check-in phải tính theo timezone property, config effective time và dấu hiệu đã gửi; dùng batch claim DB trong nhiều instance.
- Chat realtime không thay thế persistence: một message phải có seq, cursor và quyền participant; mạng đứt chỉ làm mất push, không mất lịch sử.
- Phân quyền analytics, export, hội thoại và audit phải áp dụng vào query; chỉ ẩn nút frontend không bảo vệ dữ liệu.

10. Trình tự triển khai và các mốc tích hợp

Giai đoạn	Nội dung có thể triển khai song song	Đầu ra tích hợp
DEV 01 – Identity, Catalog & Messaging	Identity/JWT/RBAC, cấu hình mật khẩu & policy, property/room/rates, MinIO, search/quote, chat và xử lý phản ánh + đề nghị check-in trên hội thoại.	UC-01→UC-08; UC-27→UC-30
DEV 02 – Booking, Inventory & Fake Bank	Booking/hold/inventory, concurrency, expiry, cancellation, check-in/out; Fake Bank + payment intents + refund/saga + đối soát và fault injection.	UC-09→UC-15; UC-35→UC-36
DEV 03 – Analytics, Email & Operations	Review/notification; dashboard và báo cáo phức tạp (PostgreSQL tuning + Redis); export; email templates, mail gateway, reminder scheduler, audit/monitoring/admin vận hành.	UC-16→UC-26; UC-31→UC-34
D. Ngoại lệ	Hết hạn, hủy, refund, orphan cleanup, webhook late/duplicate	E2E: retry, cancel, recover, reconcile
E. Vận hành	Check-in/out, reviews, notifications, reports, monitoring	Integration tests, dashboard, load tests, hướng dẫn triển khai

10.1. Quy ước tích hợp giữa các phân hệ

- Contract-first: thống nhất JSON request/response, validation, error code, timezone và enum trạng thái trước khi ghép.
- Migration theo version Flyway và ownership bảng; không để hai người sửa cùng migration đã áp dụng.
- Không gọi thẳng repository phân hệ khác từ controller; dùng application service/query API có interface rõ.
- Branch/merge nhỏ theo tính năng; CI chạy format, unit test và Testcontainers integration test trên mỗi thay đổi quan trọng.
- Test xuyên phân hệ phải được chạy lại sau sửa schema/state machine; review cả tác động đến cache, audit và báo cáo.

11. Phụ lục: cấu trúc package và quy ước bàn giao

Ưu tiên tổ chức theo feature: identity/{api,application,domain,infrastructure}; property; catalog; media; search; booking; inventory; payment; refund; stay; review; notification; reporting; audit; shared/{error,idempotency,observability,time}; config. Controller chỉ chuyển đổi giao thức, application service điều phối use case, domain giữ invariant/state transition, repository thực hiện persistence. Không tạo abstraction trung gian khi chưa mang lại giá trị rõ ràng.

- README có hướng dẫn chạy local, cấu hình Postgres/Redis/MinIO/Prometheus/Grafana, biến môi trường ví dụ và cách dựng dữ liệu mẫu.
- Tài liệu có ERD, state diagram, danh sách API theo quyền, ví dụ lỗi nghiệp vụ và transaction boundary của booking/payment/refund.
- Có Flyway migration theo phiên bản, seed data không chứa credential thật; tests bằng Testcontainers/PostgreSQL thật khi kiểm tra lock/constraint.
- Có kịch bản vận hành: mất Redis, MinIO chậm, DB rollback, webhook lặp/muộn, scheduler nhiều instance và hướng đối soát.
- Ghi nhận quyết định thiết kế (ADR ngắn): mô hình tồn kho, khóa DB, TTL cache, policy hoàn tiền, outbox và timezone.

11.1. Quy cách đặc tả BA áp dụng cho từng use case

- Mỗi use case ghi mã, mục đích, tác nhân, tiền điều kiện, kích hoạt, luồng chính theo bước, luồng thay thế và ngoại lệ, hậu điều kiện, quy tắc nghiệp vụ, dữ liệu đầu vào/đầu ra, trạng thái liên quan và tiêu chí chấp nhận.
- Tại các điểm có tiền/tồn kho/quyền, bổ sung bất biến kiểm tra được, thời điểm commit, idempotency key, mã lỗi và hành vi retry/compensation.
- Dữ liệu thống kê phải có data dictionary cho từng metric (tử số, mẫu số, grain, timestamp axis, timezone, status inclusion, data source, freshness).
- Các API có chuẩn lỗi: 400 VALIDATION_ERROR, 401 UNAUTHORIZED, 403 FORBIDDEN_SCOPE, 404 RESOURCE_NOT_FOUND, 409 IDEMPOTENCY_CONFLICT/STATE_CONFLICT, 422 BUSINESS_RULE_VIOLATION, 429 RATE_LIMITED, 503 TEMPORARILY_UNAVAILABLE.
- Các số thời gian, retry, cỡ cache, mật khẩu, giới hạn filter đều là giá trị cấu hình hoặc mục tiêu thử nghiệm; không ghi thành quy định cố định nếu nghiệp vụ chưa phê duyệt.

11.2. Phi chức năng: an toàn, tốc độ, tin cậy và kiểm tra

- Toàn hệ thống dùng Java 21, Spring Boot 3.x, Spring Data JPA/EntityManager, PostgreSQL, Redis, MinIO, Docker Compose, Actuator, Micrometer, Prometheus/Grafana; Fake Bank là service thử nghiệm độc lập.
- Đối với thống kê, cần test dataset lớn, EXPLAIN ANALYZE (BUFFERS), so sánh query trực tiếp với summary table, đo p50/p95/p99, DB CPU/IO, Redis hit ratio và cache freshness.
- Đối với thanh toán, chứng minh tính đúng đắn trước hiệu năng: debit ledger không trùng, refund không vượt amount, webhook và reconciliation eventual consistent; fault injection có kiểm tra trạng thái cuối.
- Đối với nhắn tin, yêu cầu bền vững nội dung DB, dữ liệu riêng tư theo quyền, rate limiting, index trên (conversation_id, seq), độ trễ gửi thông báo đo riêng với ghi nhận tin nhắn.
- Đối với email, có trạng thái mail event và lịch sử lần gửi, retry bounded, cảnh báo backlog, chống gửi trùng và bảo vệ biến nhạy cảm.
- Bảo mật hệ thống dùng JWT, RBAC + data scope, masking PII, audit, input validation, SSRF/path traversal protections, HMAC webhook, secret rotation và cache isolation.
- Khả năng vận hành: migration DB versioned, backup/restore, log correlationId, dashboards lỗi payment/mail/scheduler/query; không public actuator nhạy cảm.

12. Phân chia công việc cho 03 developer

12.1. Kịch bản tích hợp cuối cùng

Luồng 01. Đăng ký/đăng nhập thành công → HOST tạo property/phòng/giá/ảnh và publish.

Luồng 02. Khách tìm thấy phòng → mở chi tiết/quote → tạo hold thành công; request cạnh tranh không được bán trùng.

Luồng 03. Thanh toán success → booking CONFIRMED; callback duplicate không phát sinh hiệu ứng hai lần.

Luồng 04. Khách hủy booking đúng chính sách → refund mô phỏng hoàn tất; số liệu báo cáo cập nhật đúng.

Luồng 05. Giữ chỗ hết hạn và callback success đến muộn → không xác nhận trùng; refund/reconciliation xử lý.

Luồng 06. HOST check-in/out → khách gửi review; notifications và audit có liên kết trace; Grafana quan sát được luồng.

12.4. Luồng nghiệm chứng tích hợp bổ sung

- Khách đăng ký -> policy password động -> nhận email ACCOUNT_CREATED/VERIFY theo template version -> đăng nhập -> tìm phòng -> mở chat hỏi chủ -> HOST phản hồi.
- Khách tạo hold -> gọi Fake Bank 2 lần cùng key -> chỉ debit một lần -> webhook duplicate -> booking CONFIRMED một lần -> email xác nhận đúng một lần logical.
- Fake Bank SUCCEEDED nhưng update booking lỗi -> đối soát -> booking expired -> refund idempotent -> gửi mail REFUND_SUCCEEDED -> revenue reports phản ánh captured và refunded đúng kỳ.
- Khách gửi phản ánh -> HOST nhận tin/rep -> yêu cầu check-in 18:30 -> HOST đồng ý -> scheduler nhắc trước N giờ theo config, không gửi khi booking đã CANCELLED.

- ADMIN tạo template và publish version mới -> mail queued trước đó vẫn dùng version cũ -> retry sau SMTP timeout không sinh business event mới.
- HOST xem dashboard lọc 5–8 điều kiện -> SQL đã phân tích EXPLAIN -> Redis cache hit -> booking/refund update invalidate -> số liệu cập nhật, không lộ property của HOST khác.